Java基础问题

1. 集合

1.1 高频问题

Q1：⭐HashMap 与 ConcurrentHashMap 区别

我从核心本质、关键实现和实际使用三个层面来回答：

首先，**两者最核心的区别是线程安全性**。HashMap 是完全线程不安全的非同步集合，而 ConcurrentHashMap 是**专门为并发场景设计**的线程安全集合。这个本质差异导致了它们在底层实现、性能表现和使用限制上的所有不同。

先说说 HashMap 的线程安全问题。在多线程环境下，HashMap 会出现很多问题：JDK1.7 中，多个线程同时扩容会导致环形链表，get 操作时引发死循环，CPU 直接打满；JDK1.8 虽然修复了死循环问题，但仍然存在数据覆盖、数据丢失和不一致的情况，因为它的所有操作都没有任何同步措施。

然后是 ConcurrentHashMap 的线程安全实现，这里有一个很重要的 JDK 版本演进：

- JDK 7 采用的是**分段锁机制**，把整个哈希表分成 16 个独立的 Segment，每个 Segment 继承自 ReentrantLock，操作时只锁对应的段，这样默认最多可以有 16 个线程同时写入，大大提高了并发度。
- 到了 JDK 8，ConcurrentHashMap 做了重大改进，取消了分段锁，改用**CAS+synchronized**的组合实现，锁粒度进一步细化到了每个链表或红黑树的头节点。也就是说，只有当多个线程操作同一个桶的时候才会发生竞争，理论上并发度等于数组的长度，比 7 版本提升了很多。而且它的读操作几乎是无锁的，通过 volatile 保证可见性，性能非常接近 HashMap。

> **优化表述**：插入时，如果目标桶为空，用 CAS 直接写入头节点，无需加锁；如果桶不为空（发生哈希冲突），才对头节点加 `synchronized` 锁，处理链表或红黑树的插入。这样设计使得空桶插入完全无锁竞争。

接下来是几个关键的差异点：

第一，底层数据结构。JDK 8 之后，两者的结构基本一致，都是数组 + 链表 + 红黑树，链表长度超过 8 且数组长度超过 64 时会转红黑树，节点数少于 6 时转回链表。不同的是 ConcurrentHashMap 的 Node 节点用 volatile['vɒlətaɪl] 修饰了 value 和 next 指针，保证了多线程下的可见性。

第二，空值支持。HashMap 允许一个 null 键和多个 null 值，而 ConcurrentHashMap 不允许 null 键和 null 值。这是并发集合的通用设计，因为在并发环境下，无法区分 "key 不存在" 和 "key 对应的 value 是 null" 这两种情况，会产生歧义。

第三，扩容机制。HashMap 是单线程扩容，而 JDK1.8 的 ConcurrentHashMap 支持并发扩容，多个线程可以一起参与元素迁移，大大缩短了扩容时间。扩容过程中读操作可以正常进行，写操作只会被阻塞在正在迁移的桶上。

这里我想澄清一个常见的误区：很多人以为 ConcurrentHashMap 的所有操作都是原子的，其实不是。单个的 put、get、remove 操作是原子的，但复合操作不是，比如 "先判断再更新" 这种操作，中间可能被其他线程打断，需要额外加锁保证原子性。

最后是适用场景。如果是单线程环境，或者是方法内部的局部变量，不会被多线程访问，那肯定用 HashMap，性能最高；

> MerchantAdminChainContext.java — 责任链模式注册表
>
> 在 run() 启动时将所有 MerchantAdminAbstractChainHandler 实现类按 mark() 分组注册到此 Map。handler(mark, requestParam) 调用时通过 mark 取出对应的处理器链并依次执行。
>
> 为什么用 HashMap：
>
> > - mark 是业务标识符（非自增整数），天然适合作为 HashMap 的 key。
>
> > - 注册发生在 CommandLineRunner 的单线程启动阶段（run()），不存在并发写入竞争。
>
> > - 启动后只有读操作，无需并发写。

如果是多线程并发访问的场景，特别是读多写少的情况，强烈推荐使用 ConcurrentHashMap，它的性能远高于 Hashtable 和 Collections.synchronizedMap 这两种全表锁的实现。

Q2：Arraylist和LinkedList的区别，哪个集合是线程安全的?

关于 ArrayList 和 LinkedList 的区别，我从核心本质、关键差异、线程安全和适用场景四个方面来回答：

首先，两者最核心的本质区别是**底层数据结构不同**：ArrayList 基于动态数组实现，内存空间连续；LinkedList 基于双向链表实现，内存空间不连续。这个差异直接导致了它们在性能上的所有不同。

接下来是关键的性能差异：第一，随机访问性能。ArrayList 支持 O (1) 时间复杂度的随机访问，因为数组是连续的，可以通过下标直接计算出元素地址；而 LinkedList 不支持随机访问，需要从头或尾遍历，时间复杂度是 O (n)。第二，增删操作性能。在尾部增删时，两者都是 O (1)；在头部增删时，ArrayList 需要移动所有元素，是 O (n)，而 LinkedList 只需要修改指针，是 O (1)；在中间增删时，两者都是 O (n)，但 ArrayList 的元素移动是内存批量拷贝，比 LinkedList 的遍历和指针修改快得多。第三，内存占用。ArrayList 的内存浪费主要来自预留的容量空间，每次扩容会增加 50%；而 LinkedList 每个节点都需要存储前驱和后继指针，每个元素的内存开销更大。

这里我想澄清一个非常常见的误区：很多人以为 LinkedList 的增删一定比 ArrayList 快，其实不是。只有在头尾增删时 LinkedList 才有优势，在中间增删时，ArrayList 的性能反而更好。实际项目中，90% 以上的场景都应该使用 ArrayList。

然后是线程安全问题。**ArrayList 和 LinkedList 都不是线程安全的**。多线程环境下，它们会出现数据覆盖、元素丢失、链表断裂等问题，迭代器也会抛出 ConcurrentModificationException。

线程安全的替代方案主要有三种：~~第一种是 Vector，全表加 synchronized 锁，性能极差，已经过时了；第二种是 Collections.synchronizedList，包装原 List，所有方法加锁，性能一般，适合并发量较低的场景；~~第三种是 CopyOnWriteArrayList，采用写时复制机制，读操作无锁，性能极高，适合读多写少的并发场景，这也是现在最常用的线程安全 List 实现。

> **优化补充**：CopyOnWriteArrayList 有两个明显缺点：
>
> - **内存开销大**：每次写操作都会复制整个底层数组，如果数组很大或写操作频繁，GC 压力很大
> - **数据实时性弱**：读操作读的是旧数组的快照，写操作完成前无法读到最新数据，存在弱一致性问题，不适合对数据实时性要求高的场景

最后是适用场景总结。我在实际项目中，90% 的场景都使用 ArrayList，

> 批量发放优惠券时，收集用户 ID 列表；收集购物车项 CartItemVo 列表；收集 OrderItemEntity 列表

因为它的随机访问性能好，增删性能在大多数情况下也优于 LinkedList；只有在需要频繁在头尾增删，且不需要随机访问的场景下，才会考虑 LinkedList。多线程环境下，读多写少用 CopyOnWriteArrayList，读写均衡用 Collections.synchronizedList【并发收集可用/不可用优惠券列表CouponQueryServiceImpl.java】。

Q3. ArrayList的扩容机制说一下

关于 ArrayList 的扩容机制，我从核心本质、触发条件、具体过程和优化方法四个方面来回答：

首先，ArrayList 的核心是基于动态数组实现的，而数组的长度是固定的。当 ArrayList 中的元素数量超过底层数组的容量时，就会触发扩容：创建一个更大的新数组，将原数组的所有元素复制过去，然后更新引用。

接下来是扩容的具体过程。首先是触发条件：当向 ArrayList 添加元素时，会先检查当前元素个数加 1 是否大于底层数组的长度，如果是，就触发扩容。

然后是初始容量，这里有一个很重要的 JDK 版本差异：JDK 7 及之前是饿汉式初始化，创建 ArrayList 对象时底层数组就被初始化为长度 10；而 JDK8 及之后是懒汉式初始化，创建对象时底层数组是空的，第一次添加元素时才会真正初始化为长度 10。这个优化是为了避免创建了 ArrayList 但没有添加元素的场景下浪费内存。

然后是新容量的计算。默认情况下，新容量是原容量的 1.5 倍，计算方式是原容量加上原容量右移一位，也就是 oldCapacity + (oldCapacity>> 1)。有两个特殊情况：如果 1.5 倍仍然不够，就直接使用需要的最小容量；如果超过了 Integer.MAX_VALUE - 8，就使用 Integer.MAX_VALUE。

扩容的具体步骤是：先计算新容量，然后创建新数组，再使用 System.arraycopy () 这个 native 方法将原数组的所有元素复制到新数组中，最后更新 elementData 引用指向新数组。

这里我想特别提一下扩容的性能开销。扩容的主要开销来自于数组复制，当元素数量很大时，一次扩容可能会消耗大量时间。频繁扩容会严重影响性能，所以在实际项目中，我都会尽量提前预估元素数

量，在创建 ArrayList 时就指定初始容量，或者在添加大量元素前调用 ensureCapacity () 方法进行预扩容，这样可以完全避免或者减少扩容的次数。

最后澄清一个常见的误区：很多人以为 ArrayList 每次扩容是 2 倍，其实不是，从 JDK1.2 开始就是 1.5 倍，Vector 才是 2 倍扩容。

Q4. HashMap实现原理介绍一下?

关于 HashMap 的实现原理，我从核心本质、底层结构、核心流程和 JDK 版本差异四个方面来回答：

首先，HashMap 是基于哈希表实现的键值对存储集合，核心思想是通过哈希函数将 key 映射到数组下标，实现 O (1) 时间复杂度的快速存取。JDK1.7 及之前采用数组 + 链表结构；JDK1.8 及之后采用数组 + 链表 + 红黑树的组合结构，当链表长度超过 8 且数组长度超过 64 时，会转换为红黑树，将查询时间复杂度从 O (n) 降低到 O (log n)。

接下来是核心原理。首先是哈希函数，HashMap 先计算 key 的 hashCode，然后将 hashCode 右移 16 位再与自身异或，最后用这个结果与数组长度减 1 进行与运算得到数组下标。这样做的目的是让 hashCode 的高位也参与到下标计算中，减少哈希冲突。而且因为数组长度是 2 的幂次方，与运算等价于取模运算，但效率高得多。

然后是 put 方法的执行流程：首先判断数组是否为空，如果为空就初始化扩容；然后计算 key 对应的数组下标；如果该位置为空，直接插入新节点；如果不为空，判断 key 是否相等，相等就覆盖 value；如果不相等，判断是链表还是红黑树，分别进行插入；插入后判断链表长度是否超过 8，如果是就尝试转红黑树；最后判断元素数量是否超过阈值，如果是就扩容。

然后是扩容机制。当元素数量超过容量乘以加载因子（默认 0.75）时触发扩容。新容量是原容量的 2 倍。JDK 8 对扩容做了重大优化，不需要重新计算每个元素的哈希值，只需要判断原哈希值的最高位是 0 还是 1，就可以直接确定新下标是原位置还是原位置加原容量，大大提高了扩容效率。

最后是 JDK 版本的主要差异。JDK1.7 采用头插法插入元素，多线程扩容时会导致环形链表，引发死循环；JDK1.8 改用尾插法，修复了这个问题。另外 JDK1.8 引入了红黑树，解决了哈希冲突严重时的性能退化问题。

这里我想澄清一个常见的误区：很多人以为 JDK1.8 的 HashMap 是线程安全的，其实不是。它只是修复了死循环问题，但在多线程环境下仍然会出现数据覆盖、数据丢失等问题。多线程环境下应该使用 ConcurrentHashMap。

Q5. HashMap的扩容机制介绍一下

面试官您好，关于 HashMap 的扩容机制，我从核心本质、触发条件、JDK1.8 扩容过程、版本差异和优化方法五个方面来回答：

首先，HashMap 基于数组实现，数组长度是固定的。当元素数量超过阈值时，会创建一个容量为原容量 2 倍的新数组，并将原数组的元素重新哈希到新数组中，这个过程就是扩容。扩容的核心目的是降低哈希冲突概率，保证 O (1) 的平均查询性能。

接下来是**触发条件**。主要有两种情况：第一种是常规扩容，当元素数量超过容量乘以加载因子时触发，默认加载因子是 0.75，所以默认初始容量 16 的 HashMap，当元素达到 12 个时就会触发第一次扩

容；第二种是特殊扩容，当链表长度超过 8 但数组长度小于 64 时，会先扩容而不是转红黑树。

然后是 JDK1.8 的**扩容过程**，这也是现在的主流版本。整个过程分为三步：第一步计算新容量和新阈值，新容量是原容量的 2 倍，新阈值也是原阈值的 2 倍；第二步创建新数组；第三步迁移元素，这也是 JDK1.8 最大的优化点。

JDK1.8 不需要重新计算每个元素的哈希值，只需要判断原哈希值的最高位是 0 还是 1。如果最高位是 0，新下标就和原下标相同；如果最高位是 1，**新下标就是原下标加上原容量**。这个优化大幅提升了扩容效率。

这里我想解释两个关键的设计问题：第一，为什么加载因子默认是 0.75？这是在空间占用和查询性能之间的最佳平衡，基于泊松分布的统计结果，此时哈希冲突的概率极低。第二，为什么容量必须是 2 的幂次方？因为这样可以用位运算代替取模运算，提高下标计算效率，同时也方便了扩容时的元素迁移。

最后是优化方法。在实际项目中，我都会尽量提前预估元素数量，在创建 HashMap 时就指定初始容量，公式是初始容量等于预估元素数量除以 0.75 加 1，这样可以完全避免频繁扩容带来的性能开销。

Q6. 说一下HashMap和Hashtable、ConcurrentMap的区别

面试官您好，关于 HashMap、Hashtable 和 ConcurrentMap 的区别，我从核心本质、线程安全机制、关键特性和适用场景四个方面来回答：

首先，三者都是 Map 接口的实现类，最核心的区别在于线程安全机制和性能表现：HashMap 是完全线程不安全的，性能最高；Hashtable 是线程安全但性能极差，基于全表锁实现，已经被淘汰了；ConcurrentMap 是线程安全且高性能的并发 Map 接口，最常用的实现是 ConcurrentHashMap，是多线程环境的唯一推荐选择。

接下来是最关键的线程安全机制差异：

- Hashtable 的所有方法都加了 synchronized 关键字，相当于对整个对象加锁，同一时间只能有一个线程执行任何操作，即使是读操作也需要加锁，并发度极低，性能极差。
- 而 ConcurrentHashMap 采用了细粒度锁机制，这里有一个重要的 JDK 版本演进：JDK1.7 采用分段锁，把整个哈希表分成 16 个独立的 Segment，操作时只锁对应的 Segment，默认并发度是 16；到了 JDK1.8，取消了分段锁，改用 CAS 加 synchronized 的组合，锁粒度进一步细化到每个哈希桶的头节点，只有当多个线程操作同一个桶时才会发生竞争，理论上并发度等于数组长度，性能比 JDK1.7 提升了很多。而且它的读操作完全无锁，通过 volatile 保证可见性，性能非常接近 HashMap。

然后是几个关键的特性差异：第一，空值支持。HashMap 允许一个 null 键和多个 null 值，而 Hashtable 和 ConcurrentHashMap 都不允许 null 键和 null 值。这是因为在并发环境下，无法区分 "key 不存在" 和 "key 对应的 value 是 null" 两种情况，会产生歧义。第二，迭代器特性。HashMap 和 Hashtable 的迭代器是快速失败的，迭代过程中如果其他线程修改了集合结构，会立即抛出异常；而 ConcurrentHashMap 的迭代器是安全失败的，它遍历的是创建迭代器时的快照，迭代过程中其他线程可以修改集合，不会抛出异常，但也不能保证遍历到最新的数据。

这里我想澄清一个常见的误区：很多人以为 ConcurrentHashMap 的所有操作都是原子的，其实不是。单个的 put、get、remove 操作是原子的，但复合操作不是，比如先判断再更新这种操作，中间可能被其他线程打断。这时候应该使用 ConcurrentHashMap 提供的原子方法，比如 putIfAbsent、compute、merge 等。

最后是适用场景总结。在实际项目中，单线程环境我用 HashMap，因为它性能最高；多线程环境我用 ConcurrentHashMap，因为它线程安全且性能好；Hashtable 是历史遗留类，我从来不用。

2. 并发

高频问题：

Q1：线程的创建方式有哪些？

面试官您好，关于 Java 线程的创建方式，我从核心本质、四种主流方式和适用场景三个方面来回答：

首先，Java 中线程的本质是 `java.lang.Thread` 类的实例。创建线程主要有四种主流方式：继承 Thread 类、实现 Runnable 接口、实现 Callable 接口配合 FutureTask，以及使用线程池。其中前三种是任务的定义方式，第四种是线程的管理方式，生产环境中强烈推荐使用线程池。

第一种是**继承 Thread 类**。我们需要定义一个类继承 Thread，重写 run () 方法，然后创建实例调用 start () 方法启动线程。这种方式实现简单，但有一个很大的缺点，就是 Java 是单继承的，继承了 Thread 之后就不能再继承其他类了，而且任务和线程是耦合在一起的，不利于代码复用。

第二种是**实现 Runnable 接口**。这是最常用的方式，我们定义一个类实现 Runnable 接口，实现 run () 方法，然后把这个 Runnable 对象传给 Thread 的构造函数，再调用 start () 启动线程。这种方式解决了单继承的问题，实现了任务和线程的解耦，多个线程还可以共享同一个 Runnable 实例，非常灵活。

第三种是**实现 Callable 接口配合 FutureTask**。这种方式是 JDK1.5 引入的，主要解决了前两种方式没有返回值和不能抛出受检异常的问题。我们实现 Callable 接口的 call () 方法，它可以有返回值，也可以抛出异常。然后用 FutureTask 包装这个 Callable 对象，再传给 Thread 启动。最后通过 futureTask.get () 方法获取执行结果，不过这个方法会阻塞直到任务完成。

第四种是**使用线程池**，这也是生产环境中唯一推荐的方式。我们**通过 Executor 框架创建线程池**，然后把 Runnable 或者 Callable 任务提交给线程池执行。线程池的优势非常明显：它可以复用线程，减少线程创建和销毁带来的性能开销；可以控制并发数，避免系统资源耗尽；还可以统一管理线程的生命周期，支持定时执行、周期执行等高级功能。

以上这四种方式，都是基于Runnable接口。

> **优化补充**：线程池的创建有两种方式：一是通过 `Executors` 工厂类提供的快捷方法（如 `newFixedThreadPool`、`newCachedThreadPool`），但**生产环境不推荐**，因为它们的队列或线程数是无界的，容易导致 OOM；二是直接使用 `ThreadPoolExecutor` 构造函数，手动指定

> 核心线程数、最大线程数、队列容量、拒绝策略等七个参数，这样能精确控制资源边界，是生产中的标准做法。

这里我想特别强调一个非常重要的区别，就是 start () 方法和 run () 方法的区别。start () 方法才是真正启动一个新线程，它会调用 native 方法，由 JVM 调度执行 run () 方法；而直接调用 run () 方法只是一个普通的方法调用，不会启动新线程，会在当前线程中执行。

最后总结一下适用场景：如果是简单的测试场景，可以用继承 Thread 类；无返回值的任务推荐用 Runnable；需要返回值或者抛出异常的任务用 Callable+FutureTask；

而在生产环境中，必须使用线程池来管理线程，避免手动创建线程带来的各种问题。

> 第一种：ThreadPoolExecutor 手动创建线程池，适用于业务中需要并行处理、且任务之间有依赖关系的场景。 比如在 gulimall-product 商品详情页模块，需要在用户访问时，同时查询商品基本信息、销售属性、描述信息、基本属性和图片共 5 个数据源。这 5 个查询中，只有基本信息查完后其他 4 个才能并行展开，但 4 个之间是完全独立的。如果用传统同步方式，总耗时会近似等于 5 个查询时间之和；而用 ThreadPoolExecutor 配合 CompletableFuture 将它们编排成流水线后，总耗时只取决于最长的一条路径，性能提升非常明显。

Q2. 进程的通信方式?

进程之间地址空间相互隔离，不能直接访问，需要 IPC 进程间通信，常用一共七种：

第一，**匿名管道**，只适用于有父子亲缘关系的进程，半双工单向通信；

第二，**命名管道 FIFO**，以文件存在，无亲缘的本地任意进程都能通信；

第三，**信号**，异步通信，只做事件通知，传递信息量很小；

第四，**消息队列**，内核维护消息链表，按消息收发，支持分类读取，解耦异步；

> - **消息队列**：Java 生态中对应 RabbitMQ、Kafka、RocketMQ 等消息中间件，是分布式系统中最常用的进程解耦手段

第五，**共享内存**，效率最高，多个进程映射同一块物理内存直接读写，需要配合信号量保证同步；

第六，**信号量**，不传输业务数据，只用来做进程间的同步、互斥和限流；

第七，**Socket 套接字**，通用性最强，既可以本机通信，也能跨网络不同机器进程通信。

> - **Socket**：Java 中对应 `java.net.Socket`，是最常用的跨进程/跨机器通信方式，RPC 框架（Dubbo、gRPC）底层都是 Socket

Q3. 不同的线程之间如何通信?

面试官您好，关于 Java 线程间通信方式，我从核心定义、六大类方式和适用场景三个方面来回答：

首先，Java 线程间通信是指多个线程之间传递信息、协调执行顺序的过程。与进程间通信不同，线程共享同一地址空间，通信成本更低。主要可以分为六大类：基于 volatile 的可见性通信、基于 synchronized 的锁机制通信、基于等待唤醒的协作通信、基于 JUC 并发工具类的高级通信、基于管道流的字节流通信，以及基于 ThreadLocal 的线程私有通信。

第一种是**基于 volatile 关键字的通信**。它利用 volatile 保证共享变量的可见性，一个线程修改后其他线程立即看到。这种方式实现简单，性能极高，适合简单的状态标记和开关控制，但它不保证原子性，无法实现复杂的线程协作。

第二种是**基于 synchronized 关键字的锁机制通信**。它通过锁的互斥性保证同一时间只有一个线程执行临界区代码，同时保证可见性和有序性。这种方式适合保证多个线程对共享资源的安全访问，但只能实现互斥访问，不能主动协调线程的执行顺序。

第三种是**基于 wait/notify/notifyAll 的等待唤醒机制**。这是 Java 最基础的线程协作机制。线程调用 wait () 方法会释放锁并进入等待状态，其他线程调用 notify () 或 notifyAll () 唤醒等待的线程。这里有一个非常重要的区别，就是 wait () 和 sleep () 的区别：wait () 是 Object 类的方法，会释放锁，必须在同步块中调用；而 sleep () 是 Thread 类的方法，不会释放锁，可以在任何地方调用。等待唤醒机制的经典应用是生产者消费者模型，但手动实现容易出现信号丢失和虚假唤醒问题。

第四种是**基于 JUC 并发工具类的高级通信**，这也是生产环境中推荐使用的方式。JDK1.5 引入的 java.util.concurrent 包提供了一系列强大的工具类：CountDownLatch 是一次性倒计时门闩，适合一个线程等待多个线程完成任务；CyclicBarrier 是循环栅栏，适合多个线程互相等待到某个点再一起执行；Semaphore 是信号量，用于控制同时访问某个资源的线程数量；Exchanger 用于两个线程之间交换数据。这些工具类避免了手动实现等待唤醒机制的各种问题，功能强大且使用简单。

第五种是**基于管道流的字节流通信**。通过 PipedInputStream 和 PipedOutputStream 建立管道，一个线程写，另一个线程读。这种方式适合线程之间传递大量字节数据，但只能在两个线程之间进行单向通信。

第六种是**基于 ThreadLocal 的线程私有通信**。ThreadLocal 为每个线程提供一个独立的变量副本，每个线程只能访问自己的副本，从而实现线程隔离。这种方式适合传递线程上下文信息，比如用户登录信息、事务 ID 等。但使用完后必须调用 remove () 方法，否则会导致内存泄漏。

> **优化补充**：`ThreadLocal` 的内存泄漏根因在于其底层结构。每个线程持有一个 `ThreadLocalMap`，其中 key 是 `ThreadLocal` 的**弱引用**，value 是线程变量的**强引用**。当 `ThreadLocal` 对象本身被 GC 回收后，key 变为 null，但 value 仍被强引用无法回收，如果线程是线程池中的复用线程（长期存活），这个 value 就会一直占用内存，造成泄漏。因此用完必须调用 `remove()` 手动清除。

Q4. juc包下你常用的类?

关于 JUC 包下常用的类，我从核心作用、六大分类和生产实践三个方面来回答：

首先，JUC 是 java.util.concurrent 包的简称，是 JDK1.5 引入的官方并发编程工具包，提供了比传统 synchronized 和 wait/notify 更高效、更灵活的并发解决方案。我常用的 JUC 类主要分为六大类：锁机制类、原子操作类、并发集合类、线程池与执行器类、同步工具类和阻塞队列类。

第一类是**锁机制类**，主要用于替代 synchronized。最常用的是 ReentrantLock，它是可重入的独占锁，支持公平 / 非公平模式、可中断获取锁和多个条件变量，比 synchronized 更灵活。还有 ReentrantReadWriteLock，它分离了读锁和写锁，读 - 读不互斥，在读多写少的场景下能大幅提高并发性能。JDK1.8 还引入了 StampedLock，支持乐观读模式，性能更高。

第二类是**原子操作类**，基于 CAS 操作实现，提供了线程安全的原子操作，避免了锁的开销。常用的有 AtomicInteger、AtomicLong 用于计数器和序列号生成，AtomicBoolean 用于状态标记，还有 AtomicStampedReference 用于解决 CAS 的 ABA 问题。

第三类是**并发集合类**，这是生产环境中使用频率最高的。最常用的是 ConcurrentHashMap，它是线程安全的 HashMap，JDK1.8 采用 CAS+synchronized 实现，锁粒度很细，性能非常高。还有 CopyOnWriteArrayList，它采用写时复制的原理，读操作无锁，适合读多写少的场景，比如配置列表和白名单。

第四类是**线程池与执行器类**，这是生产环境中管理线程的唯一推荐方式。核心是 ThreadPoolExecutor，我在项目中都是手动创建 ThreadPoolExecutor，合理配置核心参数，避免使用 Executors 提供的默认线程池。JDK1.8 还引入了 CompletableFuture，支持函数式编程，能大幅简化异步代码的编写，非常适合复杂的异步任务编排。

第五类是同步工具类，用于实现线程之间的协作。最常用的有 CountDownLatch，它是一次性倒计时门闩，适合一个线程等待多个线程完成任务；CyclicBarrier 是循环栅栏，适合多个线程互相等待到某个点再一起执行；Semaphore 是信号量，用于控制同时访问某个资源的线程数量，也就是限流。

第六类是阻塞队列类，是线程池工作队列的基础，也常用于生产者消费者模型。常用的有 ArrayBlockingQueue 是有界队列，LinkedBlockingQueue 默认是无界队列，SynchronousQueue 是同步队列，不存储元素。

这里我想强调一下生产环境的最佳实践：优先使用 synchronized，只有在需要它不支持的特性时才用 ReentrantLock；简单的计数器用原子类；高并发场景下用 ConcurrentHashMap；必须手动创建线程池；优先使用 JUC 提供的同步工具类，避免手动实现 wait/notify。

> **优化补充**：`ReentrantLock` 相比 `synchronized` 多了四个能力：① 可中断等待（`lockInterruptibly()`）；② 超时尝试获取锁（`tryLock(timeout)`）；③ 公平锁模式；④ 一个锁可绑定多个 `Condition`，精确唤醒指定条件的线程。只有需要这四个特性之一时才用 `ReentrantLock`，否则 `synchronized` 更简洁，且 JVM 对它做了更多优化（锁升级机制）。

Q5. synchronized和reentrantlock及其应用场景？

面试官您好，关于 synchronized 和 ReentrantLock 的区别，我从核心本质、特性对比、底层原理和应用场景四个方面来回答：

首先，两者最核心的本质区别是：**synchronized 是 JVM 层面的原生关键字，由 JVM 自动管理锁的获取和释放；而 ReentrantLock 是 JDK1.5 引入的 API （JUC包下的）层面的锁，需要手动获取和释放锁**。它们都是独占可重入锁，用于保证共享资源的线程安全。

接下来是关键的特性对比：

- 可重入性：两者都支持可重入。synchronized 通过 Monitor 的计数器实现，ReentrantLock 通过 AQS 的 state 变量实现。

- 公平性：synchronized 只支持非公平锁，而 ReentrantLock 可以选择公平或非公平模式，默认是非公平。
- 可中断性：synchronized 获取锁的过程中不能响应中断，而 ReentrantLock 的 lockInterruptibly () 方法可以响应中断，避免死锁。
- 可超时性：synchronized 获取不到锁会一直阻塞，而 ReentrantLock 的 tryLock () 方法可以尝试获取锁一段时间，超时返回 false。
- 条件变量：synchronized 只有一个 wait set，而 ReentrantLock 可以创建多个 Condition 条件变量，实现更精细的线程协作。

然后是底层实现原理，这也是两者最大的区别之一：synchronized 基于对象监视器 Monitor 实现。JDK1.6 对 synchronized 做了重大优化，引入了锁升级机制：锁会从偏向锁开始，当有竞争时升级为轻量级锁，通过 CAS 自旋获取锁；当竞争非常激烈时，再升级为重量级锁，线程进入阻塞状态。这个升级机制大大提高了 synchronized 的性能。

而 ReentrantLock 基于 **AQS 抽象队列同步器**实现。AQS 内部维护了一个 volatile 的 state 变量表示锁的状态，还有一个双向链表的等待队列。线程通过 CAS 尝试修改 state 变量来获取锁，如果获取失败，就进入等待队列阻塞。当锁被释放时，会唤醒等待队列中的头节点线程。

关于性能，JDK1.5 之前 ReentrantLock 性能远优于 synchronized，但 JDK1.6 之后，synchronized 引入了锁升级机制，性能大幅提升。在低并发场景下，synchronized 的偏向锁和轻量级锁开销更小，性能更好；只有在高并发竞争激烈的场景下，ReentrantLock 才略有优势。

最后是应用场景。在实际项目中，我优先使用 synchronized，因为它使用简单，不需要手动释放锁，不容易出错，而且在大多数场景下性能足够。只有在需要 synchronized 不支持的特性时，我才会使用 ReentrantLock，比如需要公平锁、可中断获取锁、可超时获取锁，或者需要多个条件变量实现更精细的线程协作。

> 总结我项目中的实践： JVM 级别的 synchronized 用于解决单机内的缓存击穿、集合并发访问等简单场景；对于需要跨服务保证原子性的场景（如秒杀扣库存、优惠券领取），必须使用分布式锁（Redisson）而不是 ReentrantLock——因为多实例部署时，每个服务都有自己的 JVM 进程，ReentrantLock 只能锁住当前进程，其他进程的请求根本感知不到这把锁。

这里我想特别强调一点：使用 ReentrantLock 时，必须在 finally 块中调用 unlock () 方法释放锁，否则会导致死锁，这是一个非常容易犯的错误。

Q6. Threadlocal作用，原理，具体里面存的key value是啥，会有什么问题，如何解决?

面试官您好，关于 ThreadLocal，我从核心作用、底层原理、存在的问题和解决方案四个方面来回答：

首先，ThreadLocal 的核心作用是**实现线程数据隔离**。它为每个线程创建一个独立的变量副本，每个线程只能访问自己的副本，互不干扰，从根本上避免了多线程环境下的线程安全问题。~~很多人误以为 ThreadLocal 是用来解决共享变量问题的，这是一个常见的误解，~~它的核心是隔离而不是共享。

然后是底层实现原理，这也是 ThreadLocal 最关键的部分。很多人以为变量是存在 ThreadLocal 里面的，其实不是。**每个 Thread 对象内部都有一个 ThreadLocalMap 类型的成员变量**，真正的变量是存储在这个 ThreadLocalMap 里面的，而 ThreadLocal 只是一个工具类，用来操作 Thread 对象中的 ThreadLocalMap。

当我们调用 ThreadLocal 的 set () 方法时，它会获取当前线程的 ThreadLocalMap，然后以当前 ThreadLocal 对象为 key，传入的 value 为值，存入 Map 中。调用 get () 方法时，也是以当前 ThreadLocal 对象为 key，从当前线程的 Map 中获取对应的 value。

接下来是大家最关心的问题：ThreadLocalMap 的 key 和 value 到底是什么？ThreadLocalMap 内部维护了一个 Entry 数组，每个 Entry 是一个键值对。**key 是 ThreadLocal 对象本身，并且是弱引用；value 是线程私有的变量值，是强引用**。

这里我想特别解释一下为什么 key 要用弱引用。如果 key 是强引用，即使 ThreadLocal 对象已经没有外部引用了，也不会被 GC 回收，导致内存泄漏。使用弱引用后，当 ThreadLocal 对象没有外部强引用时，会被 GC 回收，key 就会变成 null。

但是这也带来了一个严重的问题，就是**内存泄漏**。因为 value 是强引用，不会被 GC 回收。这样 ThreadLocalMap 中就会出现很多 key 为 null、value 不为 null 的 Entry，这些 Entry 永远不会被访问到，也不会被回收，直到线程结束。如果线程是线程池中的核心线程，会一直存在，就会导致严重的内存泄漏，最终可能引发 OOM。

除了内存泄漏，ThreadLocal 还有一个问题就是**脏数据**。线程池会复用线程，当一个线程执行完任务后，如果没有清空 ThreadLocalMap，下一个任务复用这个线程时，就会读取到上一个任务留下的脏数据。

那么如何解决这些问题呢？**唯一正确的解决方案就是，使用完 ThreadLocal 后，必须手动调用 remove () 方法**，删除对应的 Entry。而且一定要在 finally 块中调用，确保即使发生异常也能执行。remove () 方法会直接从 ThreadLocalMap 中删除对应的 Entry，彻底释放内存，同时也解决了脏数据问题。

这里我想澄清一个常见的误区：很多人以为 ThreadLocal 的内存泄漏是因为弱引用，其实不是。内存泄漏的根本原因是没有手动调用 remove () 方法。如果每次使用完都调用 remove ()，就不会有任何内存泄漏问题。

最后是 ThreadLocal 的应用场景。在实际项目中，我主要用它来保存用户登录信息、事务 ID 和数据库连接等上下文信息，避免在方法之间层层传参。同时我会严格遵守最佳实践，将 ThreadLocal 声明为 private static final，并且每次使用完后都在 finally 块中调用 remove () 方法。

> **登录用户上下文：**比如在 gulimall-order 和 gulimall-cart 模块中，LoginUserInterceptor 拦截器从 Session 中取出当前登录用户的 MemberResponseVo，存入 ThreadLocal<MemberResponseVo>。后续整个请求链路中，无论是 Service 层查询用户地址（LoginUserInterceptor.loginUser.get()），还是构建订单信息、查询用户优惠券，都直接从这个 ThreadLocal 中获取，无需在每个方法之间层层传递。这在线程级别上实现了"全局变量"，非常方便。

Q7. Volatile是线程安全的吗？为什么？

面试官您好，关于 volatile 是不是线程安全的这个问题，我的答案非常明确：**volatile 不是线程安全的**。

原因很简单：线程安全需要同时满足原子性、可见性和有序性三个条件，而 volatile 只能保证其中的两个，也就是可见性和有序性，它**不能保证原子性**。

首先，我来说说 volatile 能保证什么。第一是可见性，当一个线程修改了 volatile 变量的值，JMM 会立即将这个修改刷新到主内存中，其他线程读取这个变量时，会直接从主内存中读取最新的值，而不是使用工作内存中的副本。第二是有序性，volatile 通过内存屏障禁止编译器和 CPU 对 volatile 变量的读写操作进行重排序，保证程序执行的顺序与代码的逻辑顺序一致。~~最经典的应用就是双重检查锁实现单例模式，如果没有 volatile，可能会因为指令重排序导致返回一个未初始化的对象。~~

然后是最关键的部分，为什么 volatile 不能保证原子性。原子性是指一个操作要么全部执行，要么全部不执行，中间不会被其他线程打断。volatile 只能保证单个 volatile 变量的读操作和写操作是原子的，但复合操作不是原子的。

最经典的例子就是 count++ 操作。很多人以为只要把 count 声明为 volatile 的，在多线程环境下就不会有问题，其实不然。count++ 实际上是由三个独立的步骤组成的：首先读取 count 的当前值，然后加 1，最后写回主内存。这三个步骤可能被其他线程打断。比如线程 A 和线程 B 同时读取 count 的值为 0，然后各自加 1，再写回主内存，最终结果就是 1 而不是 2，出现了数据丢失的问题。

volatile 只能保证第一步读取的是最新值，第三步写回的是最新值，但无法保证第二步和第三步之间没有其他线程修改 count 的值。这就是 volatile 不能保证原子性的根本原因。

这里我想澄清一个常见的误区：很多人以为 volatile 比 synchronized 慢，其实恰恰相反。volatile 的读操作性能与普通变量几乎相同，写操作性能也远高于 synchronized。在只有一个线程写、多个线程读的场景下，volatile 的性能优势非常明显。

最后总结一下 volatile 的适用场景。它适合那些只有一个线程写、多个线程读的场景，比如状态标记、开关控制、一次性事件通知，以及双重检查锁实现单例模式。如果涉及到多个线程写，或者复合操作，就需要使用 synchronized 或者原子类来保证线程安全。

Q8. 什么情况会产生死锁问题？如何解决？

面试官您好，关于死锁问题，我从核心定义、产生条件、常见场景、排查方法和解决方法五个方面来回答：

首先，死锁是指两个或多个线程在执行过程中，因互相等待对方持有的资源而陷入的一种永久阻塞状态。如果没有外部干预，这些线程将永远无法继续执行，会导致系统资源耗尽、服务无响应。

死锁的产生必须同时满足四个必要条件，破坏其中任何一个条件就能避免死锁。这四个条件分别是：

第一，**互斥条件**，资源在同一时间只能被一个线程持有，这是锁的基本特性，无法破坏；

第二，**请求与保持条件**，线程已经持有了至少一个资源，又请求其他被其他线程持有的资源；

第三，**不可剥夺条件**，线程已经持有的资源，在未使用完之前不能被其他线程强行剥夺；

第四，**循环等待条件**，多个线程之间形成了一个循环等待资源的链条。

最经典的死锁场景就是两个线程互相等待对方释放锁。比如线程 1 持有 lock1，然后去申请 lock2；同时线程 2 持有 lock2，然后去申请 lock1，这样两个线程就会永远等待对方释放锁。其他常见的场景还有循环等待多个资源、持有锁的同时调用阻塞方法、未正确释放锁，以及数据库死锁。

接下来是死锁的排查方法。在生产环境中，最常用的是 jstack 命令。我们先用 jps 找到 Java 进程的 PID，然后执行 jstack -l PID 命令，它会自动检测死锁并输出详细信息。~~另外，也可以使用 JConsole、VisualVM 或者 Arthas 这些可视化工具来排查死锁。~~

最后是死锁的解决方法，对应四个必要条件：第一，破坏请求与保持条件，一次性申请所有需要的资源，而不是逐步申请；第二，破坏不可剥夺条件，当线程申请资源失败时，主动释放已经持有的资源，过一段时间再重新申请；第三，破坏循环等待条件，这是最常用也最有效的方法，给所有资源指定一个全局的顺序，所有线程都按照这个顺序来申请资源；第四，使用定时锁，比如 ReentrantLock 的 tryLock () 方法，避免无限期等待锁。

~~在实际项目中，我主要通过统一锁的获取顺序和使用 tryLock () 方法来预防死锁。同时，我会尽量减少锁的数量和持有时间，避免在持有锁的同时调用阻塞方法。如果遇到死锁问题，我会使用 jstack 命令来排查定位。~~

Q9. ⭐如何创建线程池？线程池常见参数有哪些？

面试官您好，关于线程池的创建和核心参数，我从核心原则、创建方式、七大参数、工作原理和最佳实践五个方面来回答：

首先，线程池是基于池化思想管理线程的工具，核心目的是复用线程、控制并发数、统一管理线程生命周期。这里我想特别强调一个生产环境的铁律：**绝对不能使用 Executors 工具类创建线程池**，必须手动创建 ThreadPoolExecutor，明确指定所有参数，避免资源耗尽的风险。

Executors 提供的四种默认线程池都有严重问题：FixedThreadPool 和 SingleThreadPool 使用无界队列，任务堆积时会导致 OOM；CachedThreadPool 和 ScheduledThreadPool 的最大线程数是 Integer.MAX_VALUE，任务突增时会创建大量线程，耗尽 CPU 和内存。

接下来是 ThreadPoolExecutor 的七大核心参数，这也是线程池最关键的部分：第一，corePoolSize 核心线程数，是线程池中长期保持存活的线程数量。配置原则是 CPU 密集型任务设置为 CPU 核心数 + 1【因为当某个线程因页缺失或者其他原因阻塞时，额外的这个线程可以顶上去，保证CPU不闲置】，IO 密集型任务设置为 CPU 核心数 * 2【因为IO密集型的线程大部分时间在等待IO，CPU经常空闲，可以开更多线程来利用CPU】。第二，maximumPoolSize 最大线程数，是线程池允许创建的最大线程数量。第三，keepAliveTime 空闲线程存活时间，当线程数超过核心线程数时，多余的空闲线程在被销毁前的等待时间，一般设置为 60 秒。第四，unit 时间单位，一般用秒。第五，workQueue 工作队列，用于存储等待执行的任务。生产环境推荐使用 **ArrayBlockingQueue 有界队列**，能有效控制资源，绝对不能使用无界队列。第六，threadFactory 线程工厂，用于创建线程。我一般会用 ThreadFactoryBuilder 自定义线程工厂，设置线程名称前缀，方便问题排查。第七，handler 拒绝策

略，当线程池和队列都满了之后对新任务的处理策略。JDK 提供了四种，默认是 AbortPolicy 抛出异常，我比较常用的是 CallerRunsPolicy，由提交任务的线程自己执行，这样既不会丢失任务，也不会给系统造成太大压力。

> **优化补充**：四种拒绝策略完整列出：
>
> - `AbortPolicy`（默认）：抛出 `RejectedExecutionException`，任务丢失
> - `CallerRunsPolicy`：由提交线程自己执行，起到降速限流的效果
> - `DiscardPolicy`：静默丢弃新任务，无任何通知，慎用
> - `DiscardOldestPolicy`：丢弃队列中最老的任务，执行新任务，适合任务有时效性的场景

然后是线程池的工作原理。当一个任务提交给线程池时，首先判断当前运行的线程数是否小于核心线程数，如果是，就创建新的核心线程执行任务；如果不是，就把任务加入工作队列；如果队列也满了，就判断当前线程数是否小于最大线程数，如果是，就创建新的非核心线程执行任务；如果也满了，就执行拒绝策略。

最后是生产环境的最佳实践。我在项目中都是手动创建 ThreadPoolExecutor，使用 ArrayBlockingQueue 作为工作队列，自定义线程工厂设置线程名称，根据任务类型配置核心线程数和最大线程数。同时，我会监控线程池的状态，包括活跃线程数、队列大小、任务执行时间等，及时发现和解决问题。不同类型的任务我会使用不同的线程池，避免互相影响。

Q10. 关于线程池参数调优或者监控的方法

面试官您好，关于线程池的参数调优和监控，我从核心原则、参数调优方法、监控方案和最佳实践四个方面来回答：

首先，线程池调优没有万能参数，核心目标是在**吞吐量、响应时间和系统资源占用**三者之间找到最佳平衡点。调优的正确思路是先监控，后压测，小步迭代，数据说话。任何脱离业务场景和监控数据的调优都是纸上谈兵。

接下来是参数调优方法，最关键的是核心线程数、队列大小和最大线程数这三个参数：第一，核心线程数。这是线程池最重要的参数，需要根据任务类型来配置。CPU 密集型任务，比如计算、加密，核心线程数设置为 CPU 核心数 + 1；IO 密集型任务，比如数据库查询、网络调用，核心线程数设置为 CPU 核心数的 2-4 倍；混合型任务最好拆分为两个独立的线程池，分别配置参数。第二，队列大小。根据任务平均执行时间和可接受的最大等待时间来计算，一般不要超过 1000。生产环境必须使用有界队列，绝对不能使用无界队列，否则会导致 OOM。第三，最大线程数。根据系统资源上限和任务突增情况设置，一般是核心线程数的 2-4 倍，绝对不能设置为 Integer.MAX_VALUE。其他参数，空闲线程存活时间一般设置为 60 秒，拒绝策略根据业务重要性选择，我比较常用的是 CallerRunsPolicy，既能保证任务不丢失，又不会给系统造成太大压力。

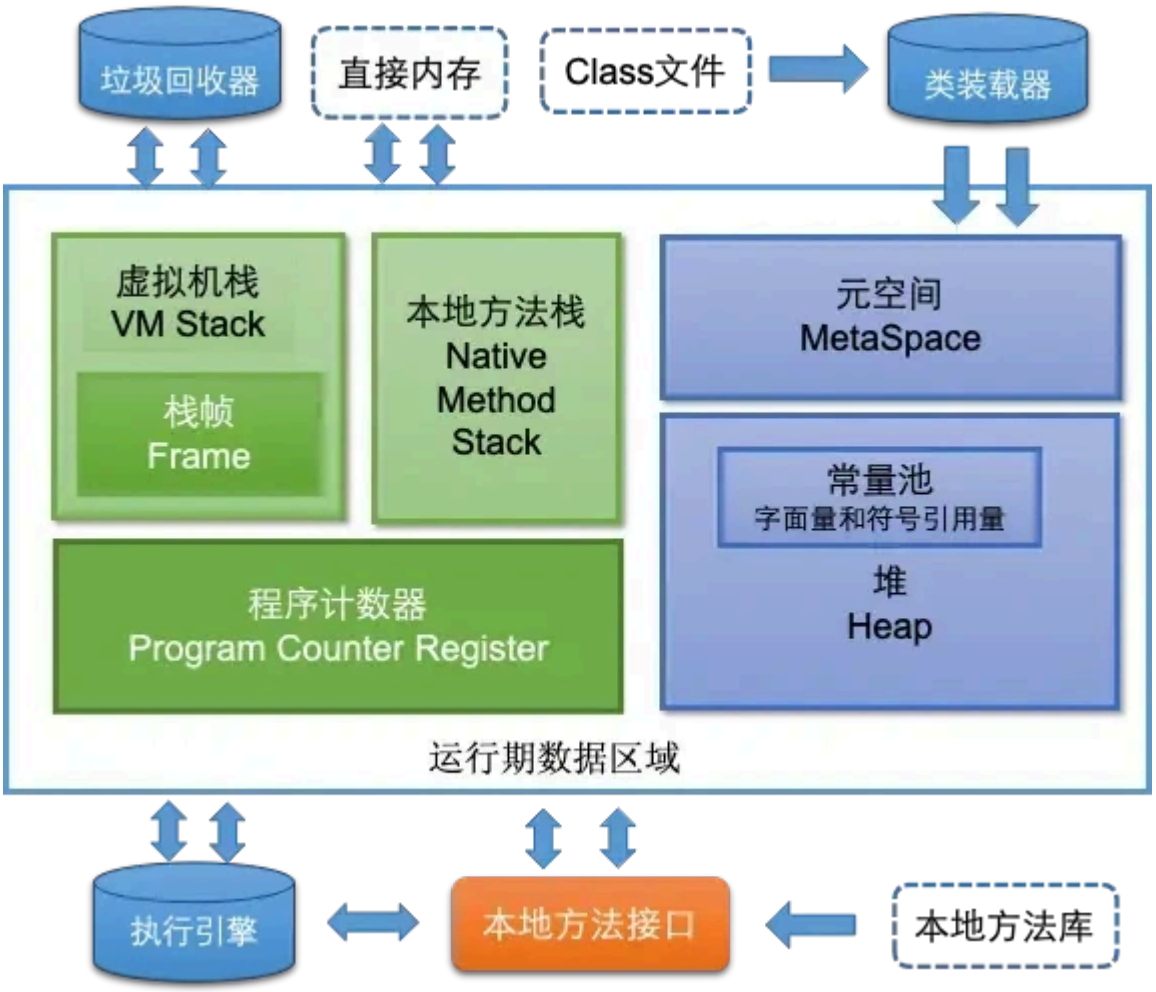
然后是**监控方法**。没有监控就没有调优，必须监控线程池的核心指标，包括活跃线程数、待处理任务数、已完成任务数、拒绝任务数、任务平均执行时间等。在开发测试环境，我会使用 jstack、jconsole 这些 JDK 自带的工具；在生产环境，我会使用 Spring Boot Actuator 暴露指标，结合

Prometheus 和 Grafana 进行可视化监控和告警。我还会自定义监控，定期采集线程池的运行指标，上报到监控系统。

最后是生产环境的最佳实践。我在项目中会遵循以下原则：不同的业务模块使用独立的线程池，避免互相影响；建立完善的监控告警体系，对核心指标设置告警阈值；预留足够的资源冗余，应对突发情况；定期压测和复盘，根据业务变化调整参数。

---

3. JVM



JVM 整体五大核心架构

VM 内部分为 5 大模块，分工明确，这是 JVM 的整体骨架：

代码块

```
1  类加载子系统
2          ↓
3  运行时数据区（内存核心）
4          ↓
```

```
执行引擎 ←→ 本地方法接口
          ↓
本地方法库
```

类加载子系统：负责把磁盘上的 .class 文件，加载到 JVM 内存里；

运行时数据区：JVM 的**内存仓库**，程序运行所有数据都存在这，最核心、面试最多；

执行引擎：负责翻译字节码、执行代码、即时编译优化；

本地方法接口 (JNI)：Java 调用 C/C++ 写的底层方法的桥梁；

本地方法库：C/C++ 写的底层系统库，比如文件操作、硬件调用。

运行时数据区

这是 JVM**内存划分**，程序运行所有变量、对象、方法都存在这里。整体分为两大类：**线程私有**、**线程共享**

核心概念

- 线程私有：每个线程独一份，线程创建就创建，线程销毁就销毁；
- 线程共享：所有线程共用，JVM 启动就创建，JVM 关闭才销毁。

线程私有区域（3 个）

3.1.1 程序计数器（Program Counter Register）

- 作用：记录当前线程执行到哪一行字节码，线程切换后能恢复继续执行；
- 特点：**唯一一个 JVM 规范没有规定内存溢出的区域**；
- 存储：当前执行字节码的行号。

3.1.2 虚拟机栈（JVM Stack）

- 每个方法执行都会创建一个**栈帧**，入栈执行，方法结束栈帧出栈；
- 栈帧里存：局部变量、方法参数、返回地址、操作数等；
- 生命周期：随线程而生，随线程而灭；
- 异常：
  - 栈深度超过限制：**StackOverflowError 栈溢出**；
  - 栈内存不够：OOM 内存溢出。

3.1.3 本地方法栈（Native Method Stack）

和虚拟机栈几乎一样，区别：

- 虚拟机栈管**Java 方法**；
- 本地方法栈管**Native 本地方法**（C/C++ 写的方法）；
- 同样会发生栈溢出和 OOM。

线程共享区域（2 个）

3.1.1 堆（Heap）【GC 主战场】

- JVM**最大的一块内存**，所有**new 出来的对象、数组**都存在堆里；
- 所有线程共享；
- **唯一目的：存对象**，也是**垃圾回收 GC 的主要工作区域**；
- 堆内部划分：**新生代 + 老年代**（后面 GC 详细讲）；
- 异常：堆内存不够 → **Java 堆 OOM**。

3.1.2 元空间（Metaspace）

替代 JDK1.8 之前的**永久代**

- 存储：类的结构信息、Class 对象、方法元数据、注解、常量池等；
- 区别永久代：元空间**使用本地物理内存**，不再占用 JVM 堆内存，基本不会溢出；
- 溢出场景：动态生成大量类、反射大量创建类 → 元空间 OOM。

总结内存区域一览表

区域	归属	存储内容	是否 GC	常见异常
程序计数器	线程私有	字节码行号	不 GC	无
虚拟机栈	线程私有	栈帧、局部变量	不 GC	SOF、OOM
本地方法栈	线程私有	本地方法栈帧	不 GC	SOF、OOM
堆	线程共享	所有 new 对象	重点 GC	OOM
元空间	线程共享	类元数据、Class	少量 GC	OOM

垃圾回收 GC

什么是 GC?

GC = Garbage Collection 垃圾回收通俗说：**JVM 自动帮我们清理堆内存中没人使用的对象，释放内存，不用程序员手动删对象。**



哪些区域会垃圾回收？

只有**堆、元空间**会 GC；虚拟机栈、本地方法栈、程序计数器**完全不用 GC**，线程结束自动释放。

什么是垃圾对象？

没有任何引用指向的对象，就是垃圾，等着被回收。或者说是

JVM 用「可达性分析」判定垃圾：从 GC Roots 出发遍历对象图，遍历不到的对象就是垃圾。

> **为什么不用引用计数法？** 引用计数法无法解决循环引用 ——A 引用 B、B 引用 A，两者计数都为 1，永远不为 0，永远收不掉。可达性分析天然免疫这个问题：从根出发，环上的对象连不上根就是垃圾，整环一起回收。

例：

代码块

```
1   User user = new User();
2   user = null; // 原来new的User对象没有引用了，变成垃圾
```

四大引用类型（由强到弱）

1. **强引用**：普通 new，只要有引用，永远不会被回收；
2. **软引用**：内存不够时才回收，适合做缓存；
3. **弱引用**：只要触发 GC 就一定回收，适合临时缓存；
4. **虚引用**：形同虚设，不能获取对象，仅用于监控对象回收。

垃圾对象怎么判定？两种算法

引用计数法

给对象加计数器，有引用 + 1，引用失效 - 1，计数器为 0 就是垃圾。**缺点**：无法解决**循环引用**（A 引用 B，B 引用 A，计数器都不为 0，永远回收不了），JVM 不用。

可达性分析算法（JVM 正在用）

核心：从 **GC Roots（根对象）** 开始往下遍历，能遍历到的就是存活对象，遍历不到的就是垃圾。

哪些是 GC Roots?

- 虚拟机栈中引用的对象；
- 静态变量引用的对象；
- 常量引用的对象；
- 本地方法栈引用的对象。

> 一句话：**GC Roots 就是程序当前正在用、必须保留的核心对象**，从它们找不到的对象，全是垃圾。

三大垃圾回收算法

3.1.1 标记 - 清除

- 步骤：标记垃圾 → 直接清除；
- 优点：简单；
- 缺点：产生**内存碎片**，大对象分配容易 OOM。

3.1.2 标记 - 复制

- 步骤：标记存活对象 → 复制到新内存区域 → 清空原区域；
- 优点：无内存碎片，效率高；
- 缺点：需要双倍内存空间；
- 适用：**新生代**。

3.1.3 标记 - 整理

- 步骤：标记存活对象 → 向一端整理压缩 → 清理末端垃圾；
- 优点：无内存碎片；
- 缺点：整理移动对象，效率低；
- 适用：**老年代**。

分代回收（JVM 默认策略）

为什么分代?

对象生命周期不一样：大部分对象朝生夕灭，少数长期存活，分开回收效率更高。

堆内存分代结构

堆 = 新生代 + 老年代

1. **新生代（占堆 1/3）**分为：Eden 区 + Survivor0 (S0) + Survivor1 (S1)默认比例：**8:1:1**
   - 新建对象优先放 Eden；
   - Eden 满了触发 Minor GC，存活对象复制到 S0/S1；
   - S 区来回复制 15 次（默认）还存活，晋升到老年代。
2. **老年代（占堆 2/3）**存放长期存活的对象、大对象；回收频率低，回收慢。

GC 分类

1. **Minor GC**：只回收新生代，频率高、速度快；
2. **Major GC**：只回收老年代；
3. **Full GC**：回收整个堆 + 元空间，**Stop The World（STW）** 暂停所有用户线程，卡顿严重，生产环境要尽量避免。

GC 类型	打扫范围	触发条件	特点
Minor GC	只扫新生代	Eden 区满了	频率极高、速度极快、无卡顿
Major GC	只扫老年代	老年代空间不足	频率低、速度慢
Full GC	扫整个堆 + 元空间	老年代满、System.gc ()	**频率极低、速度极慢、STW 卡顿**

GC 完整工作流程

我们跟着一个 `new User()` 对象，看 GC 全程怎么干活：

1. **新对象出生**对象优先放入**Eden 区**，此时 S0、S1 都是空的。
2. **第一次 Minor GC（Eden 满了）**

- Eden 区放不下新对象 → 触发 Minor GC；
- 标记存活对象，**全部复制到 S0 区**；
- 直接清空整个 Eden 区；
- 存活对象**年龄 + 1**。

3. **第二次 Minor GC**
   - Eden 又满了 → 再次 GC；
   - 把 Eden + S0 的存活对象，**全部复制到 S1 区**；
   - 清空 Eden + S0；
   - 存活对象年龄 + 1。
4. **对象轮换存活**S0 ↔ S1 来回复制，每熬过一次 Minor GC，年龄 + 1。
5. **对象晋升老年代**默认**年龄达到 15 岁**，还存活 → 直接送入**老年代**。
6. **Full GC 触发**老年代也被长寿对象占满 → 触发 Full GC，清理整个堆内存。



垃圾收集器

> 收集器就是把理论做成**能跑的、有参数的、能调优的引擎**。同一套算法可以做出多个收集器，区别在于**执行细节**：谁来做（单线程还是多线程）、何时停用户线程（STW 还是并发）、怎么组织内存（连续分代还是 Region）。

任何 GC 都在这三个目标里取舍，**三者不可兼得**：

目标	追求什么	代价
吞吐量	单位时间完成更多业务（GC 时间占比小）	每次 GC 多干活 → 停顿长
延迟（停顿）	每次 STW 尽量短、可预测	并发机制复杂、GC 频繁 → 吞吐下降
内存占用	少占内存（RSet 等结构开销小）	机制简单 → 停顿长或碎片多

无论哪个收集器，一次回收都走这六步（差异在 "每一步谁来做、做的时候用户线程停不停"）：

代码块

```
① 触发 → ② 判定（可达性分析） → ③ 标记 → ④ 处理（复制/清除/整理） → ⑤ 更新引用/指针 → ⑥ 恢复
```

收集器之间的全部差异，可以压缩成**四个维度**（这是理解所有收集器的钥匙）：

1. **线程模型**：串行（1 个 GC 线程）/ 并行（多 GC 线程但用户线程全停）/ 并发（GC 与用户线程同时跑）；
2. **内存布局**：连续分代（新生代 + 老年代连续内存）/ Region 化（不连续小块）；
3. **标记机制**：STW 全量标记 / 并发标记（三色标记 + 增量更新或 SATB）/ 染色指针标记；
4. **处理方式**：复制 / 清除（不移动，可并发）/ 整理（移动，STW）/ **并发整理**（移动但用户线程不停——ZGC/Shenandoah 的独门绝技）。

> **STW = Stop The World（世界停顿）**，指的是：**GC 执行某些阶段时，JVM 把所有应用线程（业务线程）全部暂停，只让 GC 线程干活**。暂停的这段时间就叫 "STW 停顿"。



分代组合式 vs 整堆一体式

- 老一代（Serial/ParNew/Parallel/CMS 时代）：新生代、老年代**各配一个收集器**拼起来用；
- 新一代（G1/ZGC/Shenandoah）：一套机制管整个堆，**不再需要组合**。



高频问题

Q0. JVM 的内存模型（JMM）和运行时内存结构有什么区别？

面试官您好，关于 JVM 内存模型，我从核心定义、要解决的问题、底层实现和核心规则四个方面来回答：

首先，我想先明确一个容易混淆的概念：**JVM 内存模型（JMM）和 JVM 运行时内存结构是两个完全不同的东西**。JVM 内存结构描述的是 JVM 运行时如何划分堆、栈、方法区这些内存空间，解决的是 "内存存在哪里" 的问题；而 JMM 是一套抽象的规范，解决的是 "多线程环境下如何安全访问内存" 的问题，它的核心目标是保证 Java 程序在所有平台上的并发行为一致性。

JMM 定义了线程和主内存之间的抽象关系：所有的共享变量都存储在主内存中，每个线程都有自己私有的工作内存，工作内存中存储了该线程使用的共享变量的副本。线程对共享变量的所有操作都必须在工作内存中进行，不能直接读写主内存，线程之间也无法直接访问对方的工作内存，变量值的传递必须通过主内存来完成。

基于这个模型，多线程环境下会出现三个核心问题，也就是 JMM 要解决的原子性、可见性和有序性问题：

- 原子性是指一个操作要么全部执行，要么全部不执行，JMM 通过 `synchronized` 和 `Lock` 接口来保证
- 可见性是指一个线程修改了共享变量的值，其他线程能够立即看到，JMM 通过 `volatile`、`synchronized` 和 `final` 关键字来保证
- 有序性是指程序执行的顺序按照代码的先后顺序执行，JMM 通过 `volatile` 和 `synchronized` 来禁止指令重排序

这里我想特别提一下 `volatile` 关键字，它的底层是通过内存屏障指令来实现的。在 volatile 写操作前后会插入 StoreStore 和 StoreLoad 屏障，在 volatile 读操作前后会插入 LoadLoad 和 LoadStore 屏障，这样就保证了 volatile 变量的写操作对所有线程立即可见，并且禁止了指令重排序。

然后是 JMM 最核心的 Happens-Before 原则。这个原则定义了两个操作之间的可见性保证：如果操作 A Happens-Before 操作 B，那么操作 A 的结果对操作 B 可见。**它不是指时间上的先后顺序，而是一种可见性的约定**。JMM 定义了八大 Happens-Before 规则，其中最常用的有程序次序规则、锁定规则、volatile 变量规则、线程启动和终止规则，以及传递性规则。

这里我想澄清一个常见的误区：很多人以为 `volatile` 保证原子性，其实不是。`volatile` 只保证可见性和有序性，不保证原子性。比如 `count++` 这种复合操作，即使 count 是 volatile 的，在多线程环境下仍然会出现线程安全问题，这时候就需要使用 `synchronized` 或者原子类来保证原子性。

Q2. 关于 CMS 和 G1 垃圾回收器的区别

关于 CMS 和 G1 垃圾回收器的区别，我从核心本质、回收过程、关键差异和适用场景四个方面来回答：

首先，两者最核心的本质区别是：**CMS 是基于传统分代模型的并发标记清除回收器【标记垃圾 → 直接清除】，主打低延迟；而 G1 是基于区域化分代模型的并发标记整理回收器【标记存活对象 → 向一端**

整理压缩 → 清理末端垃圾】，兼顾吞吐量和可预测的低延迟，是 JDK9 及以后版本的默认垃圾回收器。

接下来是回收过程的差异。CMS 的回收过程分为四个阶段：初始标记、并发标记、重新标记和并发清除。其中只有初始标记和重新标记会 STW，停顿时间很短，并发标记和并发清除都是和用户线程一起执行的。但 CMS 有三个明显的缺点：CPU 敏感、会产生浮动垃圾，以及标记清除算法导致的严重内存碎片。

而 G1 的回收过程也分为**四个阶段**：初始标记、并发标记、最终标记和筛选回收。G1 最大的特点是将整个堆划分为多个大小相等的 Region，每个 Region 可以是 Eden、Survivor、Old 或者专门存储大对象的 Humongous Region。在筛选回收阶段，G1 会计算各个 Region 的回收价值，根据用户指定的最大停顿时间，优先回收价值最高的 Region，这也是 G1 名字的由来。

然后是关键差异点。第一，内存模型不同，CMS 是严格的分代模型，年轻代和老年代是连续的内存空间；而 G1 是区域化分代模型，年轻代和老年代由多个不连续的 Region 组成。第二，回收算法不同，CMS 使用标记清除算法，会产生内存碎片；而 G1 整体使用标记整理算法，局部使用复制算法，几乎没有内存碎片。第三，停顿时间控制不同，CMS 无法预测停顿时间，可能出现长时间的 Full GC；而 G1 可以通过 `-XX:MaxGCPauseMillis` 参数指定最大停顿时间，实现可预测的停顿。

最后是适用场景。CMS 适合堆内存适中、对延迟要求极高的应用，比如 Web 服务器和电商系统；而 G1 适合堆内存较大、需要兼顾吞吐量和延迟的应用，是现在的主流选择。在实际项目中，我主要使用 G1 垃圾回收器，通过调整最大停顿时间参数来平衡延迟和吞吐量。



Q3. Java 中双亲委派是什么？有啥用？

关于 Java 双亲委派机制，我从核心本质、类加载器层次结构、核心工作流程、三大核心作用、破坏双亲委派的经典场景、常见误区和实际开发应用七个方面来回答：

首先，双亲委派的核心本质是Java 类加载器的一种类加载规则，它的核心思想是**向上委托、向下加载**。当一个类加载器收到类加载请求时，不会自己先加载，而是先把请求委托给父类加载器去加载，

父类加载器加载不了，子类加载器才会尝试自己加载。这个机制是 JVM 的核心安全屏障，也是 Java 能够实现跨平台和类隔离的基础。

接下来是 Java 的类加载器层次结构，这是双亲委派的基础。JVM 默认提供了三层类加载器，形成了严格的父子层级关系：**第一层是启动类加载器 Bootstrap ClassLoader**，由 C++ 实现，是所有类加载器的根，负责加载 JRE 核心类库，比如 `java.lang.*` 包下的类；**第二层是扩展类加载器** Extension ClassLoader，负责加载 JRE 扩展目录下的类库；**第三层是应用程序类加载器** Application ClassLoader，也叫系统类加载器，负责加载我们自己写的业务代码和 classpath 下的第三方 jar 包。除此之外，我们还可以自定义类加载器，继承 `ClassLoader` 类实现自己的加载逻辑。

然后是双亲委派的**核心工作流程**。当我们要加载一个 `com.example.User` 类时，首先请求会到达应用程序类加载器；它不会自己加载，而是先委托给扩展类加载器；扩展类加载器也不会自己加载，继续委托给启动类加载器；启动类加载器检查自己的核心类库中有没有这个类，发现没有，就返回给扩展类加载器；扩展类加载器检查自己的扩展目录，也没有，再返回给应用程序类加载器；最后应用程序类加载器才会在自己的 classpath 下查找并加载这个类。如果所有父类加载器都加载不了，就会抛出 `ClassNotFoundException` 异常。

接下来是双亲委派的三大核心作用，这也是它存在的意义。第一，保证核心类的安全性，防止恶意代码篡改 Java 核心类。比如有人恶意写了一个 `java.lang.String` 类，想替换 JDK 的原生 String 类，由于双亲委派机制，这个类会先委托给启动类加载器，启动类加载器会加载自己的原生 String 类，恶意类永远不会被加载，从根源上防止了核心 API 被篡改。第二，避免类的重复加载，同一个类只会被同一个类加载器加载一次，保证了 JVM 中类的唯一性。如果没有双亲委派，不同的类加载器可能会加载同一个类的多个副本，导致类型转换异常。第三，实现类的隔离，**不同的类加载器可以加载同一个类的不同版本，互不干扰**，这是 Tomcat、OSGi 等框架实现多应用隔离和模块化的基础。

然后是**破坏双亲委派的经典场景**，这也是面试必考点。双亲委派不是不可打破的规则，在一些特定场景下，为了解决实际问题，需要破坏双亲委派。第一个经典场景是SPI 服务提供者接口，比如 JDBC 驱动。JDK 的 `java.sql.Driver` 接口是由启动类加载器加载的，但是各个数据库厂商的驱动实现类是在 classpath 下的，启动类加载器加载不了，所以 JDBC 通过 `ServiceLoader` 机制，使用线程上下文类加载器来加载驱动实现类，打破了双亲委派。第二个经典场景是Tomcat 的类加载器，Tomcat 需要支持多个 Web 应用部署在同一个服务器上，每个应用可以有自己的依赖版本，所以 Tomcat 为每个 Web 应用创建了独立的类加载器，优先加载自己目录下的类，而不是委托给父类加载器，实现了应用之间的类隔离。第三个经典场景是OSGi 模块化框架，它实现了动态模块化和热部署，每个模块有自己的类加载器，类加载器之间是网状结构，而不是父子层级，完全打破了双亲委派机制。

这里我想重点澄清两个非常常见的误区：第一，很多人以为双亲委派中的 "双亲" 是指继承关系，其实不是，类加载器之间是组合关系，不是继承关系，每个类加载器内部持有一个父类加载器的引用；第二，很多人以为破坏双亲委派是不好的做法，其实不是，破坏双亲委派是为了解决特定场景下的问题，是 Java 灵活性的体现。

最后是实际开发中的应用。在项目中，我主要在两个场景下用到了自定义类加载器和双亲委派机制：第一个是热部署，通过自定义类加载器，重新加载修改后的类文件，实现不重启服务器就能更新代码，提高开发效率；第二个是依赖隔离，当项目中需要同时依赖同一个 jar 包的不同版本时，通过自定

义类加载器分别加载不同版本的类，避免版本冲突。同时，我也会注意避免在业务代码中随意破坏双亲委派机制，防止出现类加载混乱和安全问题。

Q4. 内存泄漏和内存溢出的理解？

关于 Java 内存泄漏和内存溢出，我从核心本质、两者定义与核心区别、内存泄漏常见场景、内存溢出常见场景、排查方法、最佳实践和实际开发应用七个方面来回答：

首先，两者的核心本质和关系是：内存泄漏是过程，内存溢出是结果。内存泄漏**指的是不再被使用的对象无法被垃圾回收器 GC 回收，持续占用堆内存，导致可用内存逐渐减少；**内存溢出指的是程序申请的内存超过了 JVM 可用的最大内存限制，最终抛出 `OutOfMemoryError` 异常，导致程序崩溃。内存泄漏是导致内存溢出的最主要原因，但不是唯一原因。

接下来是两者的核心区别。第一，原因不同：内存泄漏是对象生命周期管理不当，该释放的没释放；内存溢出是内存需求超过了 JVM 的承载能力。第二，表现不同：内存泄漏是渐进式的，程序运行时间越长，内存占用越高，响应越来越慢；内存溢出是突发性的，程序直接崩溃抛出异常。第三，可恢复性不同：内存泄漏在程序重启后会暂时恢复，但会再次发生；内存溢出如果是参数设置不合理导致的，调整 JVM 参数后可以解决，如果是代码问题导致的，必须修复代码。

然后是**面试必考的内存泄漏常见场景**，这也是开发中最容易踩的坑。第一，静态集合类滥用，比如用 `static` 修饰的 `ArrayList`、`HashMap`，静态集合的生命周期和 JVM 一致，会一直持有对象引用，导致对象无法被回收。第二，未关闭的资源，比如文件流、数据库连接、网络连接、Redis 连接等，这些资源如果不手动调用 `close()` 方法，底层的句柄和对象会一直占用内存。第三，监听器和回调未注销，注册了监听器但在对象销毁时没有注销，监听器会一直持有对象引用，导致内存泄漏。第四，内部类和匿名内部类，非静态内部类会隐式持有外部类的引用，如果内部类的生命周期比外部类长，就会导致外部类无法被回收。第五，ThreadLocal 使用不当，ThreadLocal 的 `ThreadLocalMap` 会持有 ThreadLocal 对象和值的引用，如果线程池的线程一直存活，而没有调用 `remove()` 方法清理，就会导致内存泄漏。第六，缓存使用不当，比如使用 HashMap 做缓存，没有设置过期时间和淘汰策略，缓存对象会一直堆积，最终导致内存泄漏。

接下来是**内存溢出的常见场景**，除了内存泄漏导致的，还有以下几种：第一，堆内存溢出，这是最常见的 OOM，原因是堆内存设置太小，或者创建了过多的大对象，比如一次性查询几十万条数据到内存中。第二，元空间溢出，JDK8 及以后用元空间代替了永久代，元空间溢出主要是因为加载的类太多，比如动态生成大量的代理类、反射生成大量的类，或者 Tomcat 部署了过多的应用。第三，栈内存溢出，主要是因为方法调用层次过深，比如无限递归，导致栈帧过多，栈内存耗尽。第四，直接内存溢出，直接内存不受 JVM 堆内存限制，但受操作系统内存限制，使用 NIO 的 `ByteBuffer.allocateDirect()` 分配直接内存时，如果分配过多，会导致直接内存溢出。

然后是内存问题的排查方法。第一，命令行工具，使用 `jps` 查看 Java 进程 ID，`jstat` 实时监控 GC 情况，`jmap` 生成堆转储快照，`jstack` 查看线程堆栈信息。第二，可视化工具，使用 Eclipse MAT 分析堆转储快照，找出占用内存最多的对象和内存泄漏的根源；使用 VisualVM 实时监控 JVM 的内存、CPU、线程等运行状态。第三，日志分析，查看 GC 日志，分析 GC 的频率和耗时，如果 Full GC 频繁且耗时很长，说明存在内存泄漏问题。

这里我想重点澄清两个非常常见的误区：第一，很多人以为 Java 有 GC 就不会有内存泄漏，这是完全错误的，GC 只能回收那些没有任何引用的对象，如果对象被无效的引用持有，GC 就无法回收，就会发生内存泄漏；第二，很多人以为内存溢出一定是内存泄漏导致的，其实不是，比如一次性加载一个 10GB 的文件到内存中，即使没有内存泄漏，也会导致堆内存溢出。

最后是实际开发中的最佳实践。在项目中，我会遵循以下原则来避免内存问题：第一，及时释放资源，使用 try-with-resources 语法自动关闭文件流、数据库连接等资源；第二，避免静态集合滥用，尽量不要使用静态集合，如果必须使用，要及时清理不再需要的对象；第三，正确使用 ThreadLocal，在使用完后一定要调用 `remove()` 方法清理；第四，合理使用缓存，使用 Redis 等分布式缓存代替本地缓存，或者使用带过期时间和淘汰策略的本地缓存，比如 Caffeine；第五，避免创建过多的大对象，大对象直接进入老年代，容易触发 Full GC，对于大数据量的处理，采用分批处理的方式；第六，合理设置 JVM 参数，根据业务需求设置合适的堆内存大小、元空间大小和 GC 收集器。同时，我会在测试环境进行压力测试，提前发现内存泄漏问题，在生产环境开启 GC 日志和监控，及时预警内存异常。

Q5. JVM内存模型里的堆和栈有什么区别?

关于 JVM 内存模型中堆和栈的区别，我从核心本质、七大核心维度对比、栈的工作机制、堆的工作机制、常见误区和实际开发应用六个方面来回答：

首先，堆和栈是 JVM 运行时数据区中最核心、最基础的两个内存区域，它们的设计目标完全不同：栈是为了方法执行服务的，负责存储方法调用的上下文和局部数据；堆是为了对象存储服务的，负责存储所有的对象实例和数组。两者配合，共同支撑 Java 程序的运行。

接下来是两者最核心的七大维度对比，这也是面试必背内容：

第一，**线程共享性不同**：栈是线程私有的，每个线程都有自己**独立的栈**空间，线程之间栈数据互不干扰，不存在线程安全问题；堆是线程共享的，整个 JVM 只有一个堆，所有线程共享堆内存，存在线程安全问题，需要通过加锁等机制保证数据一致性。

第二，**存储内容不同**：栈存储的是方法调用的栈帧，每个栈帧包含局部变量表、操作数栈、动态链接和方法返回地址，**局部变量表**中存储基本数据类型（int、long 等）和对象的引用地址；堆存储的是所有的对象实例和数组，不管是成员变量、静态变量还是局部变量指向的对象，都存储在堆中。

第三，**生命周期不同**：栈的生命周期和线程完全一致，线程启动时创建栈，线程结束时栈自动销毁，内存自动释放；堆的生命周期和 JVM 完全一致，JVM 启动时创建堆，JVM 关闭时堆才会销毁，内存需要垃圾回收器自动回收。

第四，内存大小与分配方式不同：栈的内存大小非常小，默认每个线程栈大小只有 1MB 左右，通过 `-Xss` 参数调整，内存是连续分配的；堆的内存大小是 JVM 中最大的，默认是物理内存的 1/4，通过 `-Xms` 和 `-Xmx` 参数调整，内存是不连续分配的。

第五，**垃圾回收策略不同**：栈没有垃圾回收，方法执行完毕出栈时，栈帧自动销毁，内存自动释放，不会产生垃圾；堆是垃圾回收的主要区域，99% 以上的垃圾回收都发生在堆中，采用分代回收策略，分为年轻代和老年代。

第六，异常类型不同：栈会抛出两种异常：方法调用层次过深导致的`StackOverflowError`，以及栈空间扩展失败导致的`OutOfMemoryError`；堆只会抛出一种异常：堆内存不足导致的`OutOfMemoryError: Java heap space`。

第七，分配速度不同：栈的分配速度极快，只需要移动栈顶指针即可，和汇编指令级别相当；堆的分配速度慢很多，需要在堆中查找空闲内存块，还要处理垃圾回收和内存碎片问题。

然后是栈的核心工作机制。每个方法调用都会创建一个栈帧，压入当前线程的栈顶；方法执行完毕后，栈帧自动弹出，释放内存。比如调用 A 方法，A 方法调用 B 方法，B 方法调用 C 方法，那么栈中就会依次压入 A、B、C 三个栈帧，C 方法执行完先弹出，然后是 B，最后是 A。局部变量表中的变量只在当前方法中有效，方法结束后自动失效。

接下来是堆的核心工作机制。堆按照分代收集理论划分为年轻代和老年代，默认比例是 1:2。年轻代又分为 Eden 区和两个 Survivor 区，比例是 8:1:1。新创建的对象首先分配在 Eden 区，当 Eden 区满了之后，会触发 Minor GC，存活的对象会被复制到 Survivor 区；经过 15 次 Minor GC 仍然存活的对象，会晋升到老年代。大对象和长期存活的对象会直接进入老年代，老年代满了之后会触发 Full GC，回收整个堆内存。

这里我想重点澄清两个非常常见的误区：第一，很多人以为所有对象都在堆上分配，其实不是，JVM 有栈上分配的优化技术，对于那些没有逃逸出方法的小对象，会直接分配在栈上，方法结束后自动销毁，不需要垃圾回收，大大提升了性能；第二，很多人以为栈内存越大越好，其实不是，栈内存越大，每个线程占用的内存就越多，JVM 能创建的线程总数就越少，在高并发场景下反而会导致 OOM。

Q6. 引用类型有哪些？有什么区别？

关于 Java 的引用类型，我从核心本质、四种引用类型详解、核心维度对比、典型应用场景、常见误区和实际开发应用六个方面来回答：

首先，Java 引用类型的核心本质是JVM 提供的不同强度的对象引用机制，用于精细化控制对象的生命周期和垃圾回收行为。JDK1.2 之前只有一种强引用，只要引用存在对象就不会被回收，导致很多内存敏感场景无法灵活处理；JDK1.2 之后引入了强引用、软引用、弱引用、虚引用四种引用类型，强度依次递减，让开发者可以根据业务需求选择合适的引用强度，在内存利用率和程序稳定性之间取得平衡。

接下来是四种引用类型的详细说明，这也是面试的核心考点：

第一种是**强引用** (Strong Reference)，这是 Java 默认的引用类型，我们平时用`new`关键字创建的对象都是强引用。比如`Object obj=new Object()`，只要强引用存在，垃圾回收器永远不会回收这个对象，哪怕 JVM 内存不足抛出 OOM 异常也不会回收。强引用是导致内存泄漏的最主要原因之一。

第二种是**软引用** (Soft Reference)，强度比强引用弱，需要用`SoftReference`类显式创建。它的特点是：当 JVM 内存充足时，不会回收软引用指向的对象；当 JVM 内存不足即将抛出 OOM 时，会优先回收软引用指向的对象。软引用非常适合用来实现内存敏感的缓存。

第三种是**弱引用** (Weak Reference)，强度比软引用更弱，需要用 `WeakReference` 类显式创建。它的特点是：不管 JVM 内存是否充足，**只要触发垃圾回收，就会立即回收弱引用指向的对象**。弱引用适合用来实现临时缓存，以及解决某些特定场景的内存泄漏问题。

第四种是虚引用 (Phantom Reference)，也叫幽灵引用，是最弱的一种引用类型，需要用 `PhantomReference` 类显式创建，并且必须和引用队列 `ReferenceQueue` 一起使用。它的特点是：无法通过虚引用获取到对象的实例，它的唯一作用就是跟踪对象的垃圾回收过程，当对象被回收时，虚引用会被加入到关联的引用队列中。

接下来是每种引用的典型应用场景：

- 强引用：所有普通的业务对象，比如 Service、Controller、实体类等，都是强引用。
- 软引用：图片缓存、大文件缓存，比如 Android 中的图片加载框架 Glide，就是用软引用来管理图片内存，内存不足时自动回收图片。
- 弱引用：最经典的应用是 ThreadLocal 的内部实现，ThreadLocalMap 的 key 是弱引用指向 ThreadLocal 对象，这样当 ThreadLocal 对象没有强引用时，会被 GC 回收，避免内存泄漏；另外 WeakHashMap 也是基于弱引用实现的。
- 虚引用：主要用于管理堆外内存，比如 NIO 中的 DirectByteBuffer，就是通过虚引用来跟踪堆外内存的回收，当 DirectByteBuffer 对象被回收时，会通过虚引用通知 JVM 释放对应的堆外内存。

这里我想重点澄清两个非常常见的误区：第一，很多人以为软引用在内存充足时一定不会被回收，其实不是，不同 JVM 的实现可能会有差异，有些 JVM 会在内存充足但空闲时回收部分软引用对象；第二，很多人以为 ThreadLocal 的内存泄漏是因为弱引用，其实恰恰相反，ThreadLocal 的内存泄漏是因为 value 是强引用，而不是 key 是弱引用，如果 key 是强引用，内存泄漏会更严重。

最后是实际开发中的应用。在项目中，我会根据不同的场景选择合适的引用类型：对于普通的业务对象，使用默认的强引用；对于内存敏感的本地缓存，使用软引用，内存不足时自动回收，避免 OOM；对于临时对象缓存，使用弱引用，GC 时自动清理；在使用 ThreadLocal 时，一定会在使用完后调用 `remove()` 方法，避免 value 的强引用导致内存泄漏；在涉及堆外内存操作时，会关注虚引用的使用，确保堆外内存被正确释放。同时，我会避免滥用软引用和弱引用，因为它们的回收时机不确定，可能会导致业务逻辑出现意外问题。

Q7. Java的GC机制你知道哪些?

关于 Java GC 垃圾回收机制，我从核心本质、垃圾判定算法、四大垃圾收集算法、主流垃圾收集器、GC 类型与触发时机、核心调优参数、常见误区和实际开发应用八个方面来回答：

首先，GC 的核心本质是JVM 提供的自动内存管理机制，它的核心目标是**自动回收不再被使用的对象所占用的堆内存**，避免手动内存管理带来的内存泄漏、野指针等问题，让开发者专注于业务逻辑开发。GC 主要工作在 JVM 的堆内存区域，是 Java 语言最核心的特性之一。

接下来是垃圾判定算法，也就是如何判断一个对象是否是可以被回收的垃圾。主要有两种算法：第一种是引用计数法，给每个对象添加一个引用计数器，有引用时加 1，引用失效时减 1，计数器为 0 时就

可以被回收。这种算法实现简单，但无法解决循环引用的问题，所以 Java 没有使用。第二种是**可达性分析算法**，这是 Java 使用的主流算法。它以一系列称为 "GC Roots" 的对象为起点，从这些起点开始向下搜索，搜索走过的路径称为引用链。如果一个对象到 GC Roots 没有任何引用链相连，就说明这个对象是不可达的，可以被回收。GC Roots 包括：虚拟机栈中引用的对象、本地方法栈中引用的对象、方法区中静态变量引用的对象、方法区中常量引用的对象。

然后是四大核心垃圾收集算法，这是 GC 的核心： 第一种是标记 - 清除算法，分为标记和清除两个阶段：先标记出所有需要回收的对象，然后统一清除这些对象。优点是实现简单，缺点是会产生大量内存碎片，导致大对象无法分配。 第二种是标记 - 复制算法，将内存分为大小相等的两块，每次只使用其中一块。当这块内存用完了，就将还存活的对象复制到另一块上，然后一次性清除掉已使用的那块内存。优点是没有内存碎片，分配速度快，缺点是内存利用率只有 50%。现在年轻代的 GC 都使用这种算法。 第三种是标记 - 整理算法，分为标记和整理两个阶段：先标记出所有存活的对象，然后将所有存活的对象向内存的一端移动，最后清除掉边界以外的内存。优点是没有内存碎片，内存利用率高，缺点是需要移动对象，效率较低。现在老年代的 GC 都使用这种算法。 第四种是分代收集算法，这是现在所有 JVM 默认使用的算法。它根据对象的存活周期将堆内存分为年轻代和老年代：年轻代对象存活时间短，使用标记 - 复制算法；老年代对象存活时间长，使用标记 - 整理或标记 - 清除算法。这样可以根据不同代的特点选择最合适的收集算法，大幅提升 GC 效率。

接下来是主流的垃圾收集器，这也是面试的核心考点：

- 年轻代收集器：Serial 收集器（单线程，适合客户端）、ParNew 收集器（Serial 的多线程版本，适合服务端）、Parallel Scavenge 收集器（多线程，吞吐量优先）
- 老年代收集器：Serial Old 收集器（单线程）、Parallel Old 收集器（多线程，吞吐量优先）、CMS 收集器（并发标记清除，低延迟优先）
- 全代收集器：G1 收集器（区域化分代式，兼顾吞吐量和延迟）、ZGC/Shenandoah（超低延迟，停顿时间不超过 10ms）

其中，CMS 和 G1 是目前生产环境使用最广泛的收集器。CMS 以低延迟为目标，适合互联网应用；G1 以可预测的停顿时间为目标，适合大内存应用。

然后是 GC 的类型与触发时机：

- Minor GC：年轻代垃圾回收，触发时机是 Eden 区满了，执行速度快，频率高。
- Major GC：老年代垃圾回收，触发时机是老年代空间不足，执行速度慢，频率低。
- Full GC：整个堆的垃圾回收，触发时机包括老年代满了、元空间满了、调用 System.gc ()、Minor GC 晋升失败等，会导致长时间的停顿，是调优的重点。

接下来是核心调优参数：

- 堆内存参数：`-Xms` 初始堆内存，`-Xmx` 最大堆内存，建议两者设置为相同值。
- 新生代参数：`-Xmn` 新生代大小，`-XX:SurvivorRatio` Eden 区与 Survivor 区的比例，默认 8:1:1。
- 收集器参数：`-XX:+UseConcMarkSweepGC` 使用 CMS 收集器，`-XX:+UseG1GC` 使用 G1 收集器。

- GC 日志参数：`-XX:+PrintGCDetails` 打印 GC 详细日志，`-Xloggc:gc.log` 将 GC 日志输出到文件。

这里我想重点澄清两个非常常见的误区：第一，很多人以为调用 `System.gc()` 会立即触发 Full GC，其实不是，它只是建议 JVM 进行 GC，JVM 不一定会立即执行；第二，很多人以为 GC 停顿时间越短越好，其实不是，停顿时间短意味着 GC 频率会变高，总吞吐量会下降，需要根据业务需求在延迟和吞吐量之间做平衡。

最后是实际开发中的应用。在项目中，我会根据业务特点选择合适的垃圾收集器：对于低延迟要求的互联网应用，使用 CMS 或 G1 收集器；对于高吞吐量的后台应用，使用 Parallel 收集器。然后根据业务需求合理设置堆内存大小和新生代比例，避免新生代过小导致对象过早晋升到老年代。同时，我会在生产环境开启 GC 日志，定期分析 GC 日志，监控 GC 的频率和停顿时间，如果发现 Full GC 频繁或停顿时间过长，就使用 MAT 工具分析堆转储快照，找出内存泄漏的根源，进行针对性优化。

Q8. 垃圾回收器有哪些?

面试官您好，关于 Java 垃圾回收器，我从核心本质、整体分类体系、分代收集器详解、全代收集器详解、主流收集器对比、JDK 版本演进、常见误区和实际开发应用八个方面来回答：

首先，垃圾回收器的核心本质是垃圾回收算法的具体工程实现，不同收集器针对不同业务场景做了针对性优化，核心目标是在吞吐量、延迟和内存占用三者之间取得最佳平衡。没有万能的垃圾回收器，只有最适合特定业务场景的收集器。按照工作区域和技术演进，垃圾回收器可以分为分代收集器和全代收集器两大类。

接下来是分代收集器，这是 JDK8 及之前的主流实现，严格按照年轻代和老年代划分，不同代使用不同的收集器：

年轻代收集器（均基于标记 - 复制算法）

1. Serial 收集器：最基础、最古老的单线程收集器。特点是 GC 时会暂停所有用户线程（Stop-The-World，STW），只用一个线程完成垃圾回收。优点是实现简单、内存开销小、没有线程切换开销；缺点是停顿时间长。适用于客户端应用和内存较小的嵌入式设备，是 JDK8 之前客户端模式的默认年轻代收集器。
2. ParNew 收集器：Serial 收集器的多线程版本，除了使用多线程进行 GC 外，其他行为和 Serial 完全一致。优点是在多核 CPU 环境下性能比 Serial 好很多；缺点是单核 CPU 下性能不如 Serial。它是目前唯一能和 CMS 收集器配合使用的年轻代收集器，曾经是互联网应用的首选。
3. Parallel Scavenge 收集器：也叫吞吐量优先收集器，同样是多线程的标记 - 复制算法。和 ParNew 的核心区别是它的优化目标是吞吐量（用户代码运行时间 / 总运行时间），而不是停顿时间。它可以自动调节堆内存大小和新生代比例，以达到最大吞吐量。是 JDK8 默认的年轻代收集器，适用于后台批处理、大数据计算等对吞吐量要求高的场景。

老年代收集器

1. Serial Old 收集器：Serial 收集器的老年代版本，单线程，基于标记 - 整理算法。主要作为 CMS 收集器的后备方案，当 CMS 发生 Concurrent Mode Failure 时使用。
2. Parallel Old 收集器：Parallel Scavenge 收集器的老年代版本，多线程，基于标记 - 整理算法。和 Parallel Scavenge 配合使用，组成 "吞吐量优先" 的收集器组合，是 JDK8 默认的老年代收集器。
3. CMS 收集器：全称 Concurrent Mark Sweep，并发标记清除收集器，是第一个真正意义上的低延迟收集器。它的核心目标是最短停顿时间，基于标记 - 清除算法，大部分 GC 阶段可以和用户线程并发执行。工作过程分为四个阶段：初始标记（STW，很短）、并发标记（和用户线程并发）、重新标记（STW，较短）、并发清除（和用户线程并发）。优点是停顿时间极短；缺点是会产生内存碎片、CPU 占用率高、会出现 Concurrent Mode Failure。JDK9 中被标记为废弃，JDK14 中被彻底移除。

然后是全代收集器，这是现在的主流方向，打破了年轻代和老年代的物理划分，将堆划分为多个大小相等的独立区域，同时管理整个堆：

1. G1 收集器：全称 Garbage-First，是 JDK9 及以后的默认收集器。它将堆划分为 2048 个大小相等的 Region，每个 Region 可以是 Eden 区、Survivor 区或老年代区。G1 的核心特点是可预测的停顿时间模型，可以指定最大停顿时间，它会优先回收垃圾最多的 Region。工作过程分为：初始标记、并发标记、最终标记、筛选回收。优点是兼顾吞吐量和延迟、没有内存碎片、支持大内存；缺点是小内存场景下性能不如 CMS。适用于堆内存大于 8GB 的服务端应用，是目前生产环境使用最广泛的收集器。
2. ZGC 收集器：JDK11 引入的实验性收集器，JDK15 正式转正（注意：ZGC 转正后也从未成为 JDK 的默认收集器）。它是一款超低延迟的全代收集器，基于染色指针和读屏障技术，最大停顿时间不超过 10ms，而且停顿时间不会随着堆内存的增大而增加。ZGC 几乎所有阶段都可以和用户线程并发执行，STW 时间只和 GC Roots 的数量有关。适用于对延迟要求极高的互联网应用、金融交易系统等。
3. Shenandoah 收集器：RedHat 开发的开源收集器，和 ZGC 类似，也是超低延迟的全代收集器，最大停顿时间不超过 10ms。它基于 Brooks 指针技术实现并发整理，和 ZGC 的主要区别是技术实现不同。

接下来是主流收集器的核心对比：

收集器	适用代	核心目标	最大停顿时间	适用内存	JDK 版本
Parallel 组合	分代	吞吐量优先	较长	4-8GB	JDK8 默认
CMS+ParNew	分代	低延迟	几十毫秒	4-16GB	JDK8 推荐，JDK9 废弃
G1	全代	可预测停顿	几百毫秒	8-100GB	JDK9 + 默认
ZGC	全代	超低延迟	<10ms	8GB 以上	JDK15 转正（从未成为默认）

然后是 JDK 版本的核心演进：JDK8 默认 Parallel Scavenge+Parallel Old；JDK9 默认 G1，废弃 CMS；JDK11 引入 ZGC；JDK15 ZGC 和 Shenandoah 正式转正；JDK17 默认仍是 G1（ZGC 从未成为 JDK 默认收集器）。

这里我想重点澄清两个非常常见的误区：第一，很多人以为 G1 只适合大内存，其实现在 G1 在 8GB 以上的内存环境下性能就已经超过 CMS 了；第二，很多人以为 ZGC 没有停顿，其实 ZGC 也有 STW，只是停顿时间极短，而且和堆大小无关。

最后是实际开发中的应用。在项目中，我会根据业务特点选择合适的收集器：对于 JDK8 的老项目，如果是后台批处理系统，使用默认的 Parallel 组合；如果是互联网应用，使用 ParNew+CMS 组合；对于 JDK9 及以上的新项目，优先使用 G1 收集器；对于对延迟要求极高的金融、电商核心系统，升级到 JDK17 使用 ZGC 收集器。同时，我会根据业务需求设置合理的堆内存大小和最大停顿时间，开启 GC 日志，定期分析 GC 性能，进行针对性调优。

Q9. minorGC、majorGC、fullGC的区别，什么场景触发full GC

关于 MinorGC、MajorGC 和 FullGC 的区别，我从核心本质、三者核心维度对比、各自触发时机、FullGC 七大触发场景、常见误区和实际开发应用六个方面来回答：

首先，这三种 GC 是分代收集理论下的产物，**核心区别是回收的内存区域和范围不同**。分代收集将堆内存划分为年轻代和老年代，不同代的对象生命周期不同，因此采用不同的回收策略和频率。三者的执行速度、停顿时间和对业务的影响差异巨大，其中 FullGC 是调优的核心重点，也是导致系统卡顿的最主要原因。

接下来是三者的核心维度对比，这是面试必背内容：

指标	MinorGC (年轻代 GC)	MajorGC (老年代 GC)	FullGC (全堆
回收区域	仅年轻代 (Eden+Survivor)	仅老年代	整个堆 (年轻代 + 老年代
执行速度	极快，毫秒级	较慢，几十到几百毫秒	最慢，几百毫秒到几秒
触发频率	非常高，几分钟甚至几秒钟一次	较低，几小时甚至几天一次	最低，正常情况下几小
停顿时间	极短，几乎无感知	较长，可能有轻微卡顿	最长，会导致系统明显
对业务影响	几乎无影响	影响较小	影响极大，可能导致服

然后是各自的基础触发时机：

- MinorGC：当 Eden 区空间满了的时候触发，这是最常见的 GC 类型。新创建的对象首先分配在 Eden 区，当 Eden 区满了，就会触发 MinorGC，将存活的对象复制到 Survivor 区，清空 Eden 区。
- MajorGC：当老年代空间满了的时候触发，专门回收老年代的垃圾。注意：MajorGC 不是一个标准术语，不同 JVM 实现对它的定义不同，很多时候 MajorGC 会和 FullGC 混淆，现在行业内更倾向于用 FullGC 来指代全堆回收。

- FullGC：回收整个堆内存和元空间，是最重量级的 GC，也是调优的核心目标。

接下来是面试必考的 FullGC 七大触发场景，这也是开发中最容易踩的坑：

1. 老年代空间不足：这是最常见的触发原因。当对象从年轻代晋升到老年代时，老年代没有足够的空间容纳，就会触发 FullGC。常见原因包括：新生代设置过小导致对象过早晋升、创建了过多的长生命周期对象、内存泄漏导致老年代被占满。
2. 元空间 (Metaspace) 不足：JDK8 及以后，永久代被元空间取代，当加载的类过多、动态生成大量代理类或反射生成大量类时，元空间会被占满，触发 FullGC。
3. 显式调用 `System.gc()`：代码中手动调用这个方法会建议 JVM 执行 FullGC，虽然 JVM 不一定会立即执行，但大概率会触发。生产环境中绝对禁止显式调用这个方法。
4. MinorGC 晋升失败：当 MinorGC 后，存活的对象需要复制到 Survivor 区，但 Survivor 区空间不足，对象只能直接晋升到老年代，如果老年代也没有足够的空间，就会触发 FullGC。
5. 大对象直接进入老年代：JVM 有一个参数 `-XX:PretenureSizeThreshold`，大于这个值的对象会直接分配在老年代。如果频繁创建大对象，会导致老年代快速被占满，触发 FullGC。
6. CMS 收集器的 Concurrent Mode Failure：CMS 在并发清除阶段，用户线程同时在运行，会产生新的垃圾，如果此时老年代空间不足，就会发生 Concurrent Mode Failure，CMS 会退化为 Serial Old 收集器，触发一次 FullGC，停顿时间会大幅增加。
7. G1 收集器的 Mixed GC 升级：G1 默认会优先执行 Mixed GC，回收部分老年代 Region。如果 Mixed GC 的速度赶不上对象晋升的速度，老年代空间被占满，就会触发 FullGC。

这里我想重点澄清三个非常常见的误区：第一，MajorGC 不等于 FullGC，MajorGC 只回收老年代，而 FullGC 回收整个堆和元空间，现在很多资料会把两者混用，但严格来说是不同的；第二，调用 `System.gc()` 一定会触发 FullGC，其实不是，它只是给 JVM 一个建议，JVM 可以选择忽略，但在大多数情况下会触发；第三，FullGC 只会回收老年代，其实 FullGC 会先执行一次 MinorGC，再执行老年代回收，最后回收元空间，这样可以提高回收效率。

最后是实际开发中的应用。在项目中，我会通过以下方式避免频繁 FullGC：第一，合理设置堆内存大小和新生代比例，新生代一般设置为堆内存的 1/3 到 1/2，避免新生代过小导致对象过早晋升；第二，避免创建过大的对象和数组，对于大数据量的处理采用分批处理的方式；第三，绝对禁止在代码中显式调用 `System.gc()`；第四，合理设置元空间大小，避免动态生成过多的类；第五，开启 GC 日志，定期分析 GC 日志，监控 FullGC 的频率和停顿时间；第六，如果使用 CMS 收集器，要预留足够的老年代空间，避免发生 Concurrent Mode Failure；如果使用 G1 收集器，要合理设置最大停顿时间，避免 Mixed GC 升级为 FullGC。

Spring

> Spring 是一个容器，替你把对象的『生老病死』全管了

代码块

```
                ┌──────────────────────────────────────────────────┐
                │                Spring 容器                       │
                │                                                  │
    IOC 思想 ──┼──► 容器负责对象的"创建 + 组装"（DI 注入）          │
    AOP 机制 ──┼──► 容器给对象"套代理"（织入横切逻辑）              │
    生命周期 ──┼──► 容器按固定流程"走完对象的一生"                  │
    事务/自动配置 ─► 建立在上面三者之上的应用                       │
                └──────────────────────────────────────────────────┘
```

IOC 管 "怎么造"，AOP 管 "怎么增强"，生命周期管 "造的流程"，事务 = AOP 的一种应用，自动配置 = 启动时按条件批量造 Bean

IOC 的六步（背这个流程）

```
资源定位(扫描注解/配置) → BeanDefinition 解析(类名/作用域/依赖元数据)
→ Bean 注册(进 BeanFactory 注册表) → 实例化(反射 new)
→ 依赖注入(构造器/setter/字段反射赋值) → 生命周期管理
```



Spring 事务 = Spring 框架把 "开启事务 → 执行业务 → 提交 / 回滚" 这套样板代码接管过来，让你只写一个 `@Transactional` 注解，方法就自动具备事务能力。

为什么需要Spring事务?

代码块

```
1   Connection conn = dataSource.getConnection();
2   try {
3       conn.setAutoCommit(false);          // ① 手动关自动提交 = 开启事务
4       stmt.executeUpdate("扣钱...");       // ② 业务 SQL
5       stmt.executeUpdate("加钱...");
6       conn.commit();                      // ③ 全部成功 → 提交
7   } catch (Exception e) {
8       conn.rollback();                    // ④ 出错 → 回滚
9   } finally {
10      conn.setAutoCommit(true);
11      conn.close();
12  }
```

Spring 事务的两种形式

形式	用法	特点
声明式事务（主流）	@Transactional 注解在方法 / 类上	AOP 自动拦截，开箱即用
编程式事务	TransactionTemplate.execute(...)	手动控制，灵活但代码侵入

声明式事务

```
@Service
public class OrderService {

    @Transactional
    public void placeOrder() {      // 方法上的注解
        // ① 扣库存
        // ② 生成订单
        // ③ 核销优惠券
    }
}
```

Spring 内部发生的事（四个步骤）：

代码块

```
① 代理拦截：placeOrder 被 AOP 代理包了一层（TransactionInterceptor 拦截器）
② 开启事务：方法进入前，事务管理器做三件事：
      - 从连接池拿一个 Connection
      - setAutoCommit(false)（等于 begin）
      - 把"连接 + 事务状态"绑定到当前线程的 ThreadLocal（后续 SQL 都走这个连接）
③ 执行业务：方法里的 SQL 通过 MyBatis/JDBC 执行，取到的都是 ThreadLocal 里那个连接
④ 结束收尾：
      - 方法正常返回 → commit()（提交）
      - 方法抛异常 → rollback()（回滚）+ 解除 ThreadLocal 绑定 + 归还连接
```

关键点 ①：事务的本质是 "一个连接 + setAutoCommit (false)"。Spring 事务管理器的全部工作就是管理这个连接的声明周期 —— 开启、提交、回滚、释放。

关键点 ②：连接绑定在当前线程的 ThreadLocal 里。这就是为什么事务方法里的所有 DAO 操作天然在同一个事务里 —— 它们拿到的都是同一个连接。**这也是传播行为的技术基础**（下面讲）。

关键点 ③：回滚规则（高频追问）。Spring 默认**只对 RuntimeException 和 Error 回滚，受检异常（Checked Exception，如 IOException）默认不回滚**—— 因为受检异常在 Spring 看来是 "业务可恢复" 的。想让受检异常也回滚：`@Transactional(rollbackFor = Exception.class)`。

关键点 ④：事务管理器（PlatformTransactionManager）。Spring 事务的入口是 `PlatformTransactionManager` 接口，常见实现：

- `DataSourceTransactionManager`（JDBC/MyBatis 单数据源）；
- `JpaTransactionManager`（JPA）；
- `JtaTransactionManager`（分布式）。

面试答 "Spring 事务底层" 就提：**AOP 拦截 + PlatformTransactionManager 管理 Connection + ThreadLocal 绑定资源**，这三句话是核心。

Q1. 说一下你对 Spring 的理解

我对 Spring 的理解是：它本质上是一个**轻量级的企业级开发容器和框架**，它的核心价值在于把 Java 开发中那些重复的、底层的、非业务的工作全部接管过来——对象创建、依赖注入、事务管理、生命周期管理——让开发者只关注业务逻辑本身。

Spring 的两大核心特性，这也是 Spring 的灵魂所在。

> 第一个是**IOC 控制反转**，也叫依赖注入 DI。传统开发中，对象的创建和依赖关系的管理都是由开发者自己在代码中硬编码实现的；而在 Spring 中，对象的创建、依赖注入、生命周期管理全部交给 Spring 容器来负责。开发者只需要通过配置或者注解声明依赖关系，容器会自动将依赖注入到需要的对象中，彻底解决了组件之间的紧耦合问题。
>
> 第二个是**AOP 面向切面编程**，它允许我们将那些与业务逻辑无关但又遍布在业务代码中的横切关注点，比如日志记录、事务管理、权限控制、性能监控等，从业务代码中抽离出来，统一封装成切面，然后通过动态代理的方式在运行时织入到业务代码中。这样既保持了业务代码的纯净，又实现了通用功能的复用。

接下来是 Spring 的完整生态体系，现在的 Spring 已经不是一个单一的框架，而是一个庞大的生态系统。核心是 Spring Framework，在此基础上衍生出了一系列子项目：**Spring Boot** 用于快速开发独立的生产级应用，自动配置、开箱即用；**Spring Cloud** 用于构建微服务架构，提供了服务注册发现、配置中心、熔断降级、网关路由等一整套微服务解决方案；还有**Spring Data**用于简化数据访问、Spring Security用于安全认证授权、Spring Batch用于批处理等，几乎覆盖了 Java 企业级开发的所有场景。

这里我想重点讲一下 Spring 的核心优势。第一，极致的解耦能力，通过 IOC 容器实现了组件之间的低耦合，系统的可扩展性和可维护性大大提升。第二，声明式事务管理，只需要一个 `@Transactional` 注解，就可以轻松管理事务，不用手动编写事务开启、提交、回滚的代码。

> 但这里有一个我在项目中踩过的坑：本地事务在分布式场景下的局限性。订单服务调用仓储服务锁定库存，如果库存服务超时抛异常，订单服务的事务回滚了，但库存服务的锁定操作不会回滚——这就是分布式事务问题。**RocketMQ 可靠消息 + 最终一致性：**当前生产使用的方案——订单创建成功后发一条 RocketMQ 消息给仓储服务解锁库存，库存服务执行失败后发消息回补，两边最终达到一致状态。这是一种可靠消息最终一致性的分布式事务方案，比 Seata 的性能更好，适合高并发秒杀场景

第三，**强大的集成能力**，Spring 可以无缝集成几乎所有主流的开源框架，比如 MyBatis、Hibernate、Redis、RabbitMQ 等。~~第四，完善的测试支持，Spring 容器对单元测试和集成测试提供了非常好的支持，可以很方便地进行测试。~~

学 Spring 不是学一个框架，而是学一套思想：IOC 思想、AOP 思想、扩展点思想、约定大于配置的思想。

Q2. IOC和AOP是通过什么机制来实现的?

关于 Spring IOC 和 AOP 的实现机制，我从核心本质、IOC 底层实现原理、AOP 底层实现原理、关键设计细节、常见误区和实际开发应用六个方面来回答：

首先，IOC 和 AOP 是 Spring 框架的两大灵魂特性，它们的核心本质是通过设计模式和字节码技术，实现组件解耦和横切关注点分离。IOC 控制反转解决了业务组件之间的紧耦合问题，底层基于**工厂模式 + 反射机制**实现；AOP 面向切面编程解决了横切关注点代码分散的问题，底层基于**动态代理技术**实现。两者配合，让 Java 代码更加简洁、灵活、易于维护。

接下来是 IOC 控制反转的底层实现原理。IOC 的核心是将对象的创建、依赖注入和生命周期管理全部交给 Spring 容器，开发者只需要声明依赖关系。整个实现过程分为六个核心步骤：第一是资源定位，Spring 扫描 classpath 下的配置文件或者注解，找到需要管理的 Bean；第二是Bean 定义解析，将扫描到的类信息解析成 BeanDefinition 对象，包含 Bean 的类名、作用域、依赖关系等所有元数据；第三是Bean 注册，将 BeanDefinition 注册到 BeanFactory 的注册表中；第四是Bean 实例化，容器根据 BeanDefinition，通过反射机制创建 Bean 对象；第五是依赖注入，容器自动解析 Bean 之间的依赖关系，通过构造器、setter 方法或者字段注入的方式，将依赖对象注入到目标 Bean 中；第六是生命周期管理，执行 Bean 的初始化方法、销毁方法，管理 Bean 的整个生命周期。

这里我想重点讲一下 IOC 的两个核心细节：第一是依赖注入的三种方式，构造器注入是 Spring 推荐的方式，强制依赖不可变，便于测试；~~setter 注入用于可选依赖；字段注入虽然简洁，但不推荐，因为不利于测试和依赖可见性。~~第二是Bean 的作用域，默认是单例 singleton，容器中只有一个实例；还有 prototype 原型，每次请求创建一个新实例，以及 request、session 等 web 相关作用域。

然后是 AOP 面向切面编程的底层实现原理。AOP 的核心是将与业务无关的横切关注点抽离成切面，在运行时动态织入到业务代码中。首先是 AOP 的核心概念：切面是横切关注点的封装，切点定义了哪些方法需要被拦截，通知定义了拦截前后要执行的逻辑，连接点是程序执行的具体位置，织入是将切面应用到目标对象的过程。AOP 的底层基于两种动态代理技术：第一种是JDK 动态代理，基于接口实现，要求目标类必须实现接口，通过生成接口的实现类代理对象来拦截方法；第二种是CGLIB 动态代理，基于继承实现，不需要目标类实现接口，通过生成目标类的子类代理对象来拦截方法。Spring AOP 会自动选择代理方式，默认情况下，如果目标类实现了接口，就用 JDK 动态代理，否则用 CGLIB 动态代理，也可以通过配置强制使用 CGLIB。

接下来是 AOP 的通知类型，共有五种：前置通知在方法执行前执行，后置通知在方法执行后执行，返回通知在方法正常返回后执行，异常通知在方法抛出异常后执行，环绕通知可以完全控制方法的执行，是最强大的通知类型。

这里我想重点讲两个关键设计细节：第一是IOC 的三级缓存机制，专门用来解决单例 Bean 的循环依赖问题，通过提前暴露半成品对象，让循环依赖的 Bean 能够互相引用。第二是Spring AOP 和 AspectJ 的区别，Spring AOP 是运行时织入的动态代理，只能拦截 Spring 容器管理的 Bean 的方法；AspectJ 是编译时织入的静态代理，功能更强大，但需要额外的编译器支持。

最后是实际开发中的应用。在项目中，我会优先使用构造器注入来管理依赖，保证依赖的不可变性和可测试性；使用 Spring AOP 来实现日志记录、权限控制、性能监控和事务管理这些横切关注点，避免业务代码中出现大量重复的通用逻辑；同时会注意避免循环依赖，尽量通过重构代码来解决，而不是依赖三级缓存；对于复杂的切面逻辑，会考虑使用 AspectJ 来实现。

> 环绕通知：统一异常处理，Redis 故障时抛 RRException；记录操作日志、耗时【项目中】

Q3. 依赖注入了解吗？怎么实现依赖注入的？

关于 Spring 依赖注入，我从核心本质、核心概念、三种注入方式对比、Spring 实现完整流程、关键设计细节、常见误区和实际开发应用六个方面来回答：

首先，依赖注入（DI）是IOC 控制反转思想的具体实现方式，它的核心本质是将对象之间的依赖关系从代码中硬编码转移到外部容器来管理。传统开发中，我们需要在类内部通过 `new` 关键字创建依赖对象，导致组件之间紧耦合；而依赖注入让对象只需要声明自己需要什么依赖，容器会自动将依赖对象"注入"到目标对象中，彻底实现了组件之间的解耦，提高了代码的可测试性和可维护性。

接下来是核心概念。简单来说，依赖就是一个对象正常工作所需要的其他对象，比如 UserService 依赖 UserDao；注入就是容器创建并提供这些依赖对象的过程。IOC 是一种设计思想，而依赖注入是实现这种思想的最主要方式，Spring 中几乎所有的组件解耦都是通过依赖注入实现的。

然后是三种核心注入方式的对比，这也是面试必考点。第一种是**构造器注入**，通过类的构造方法传入依赖对象，这是 Spring 官方推荐的注入方式。优点是：强制依赖不可变，保证依赖不为 null；便于单元测试；可以确保对象在实例化后就处于可用状态；能够有效避免循环依赖问题。第二种是**setter 方法注入**，通过 setter 方法传入依赖对象，主要用于可选依赖。优点是：依赖可以在对象创建后动态修改；灵活性更高。第三种是**字段注入**，直接在字段上添加 `@Autowired` 注解，不需要构造器和 setter 方法。优点是代码最简洁，但缺点非常明显：依赖不可见，不利于测试；无法保证依赖不可变；容易导致循环依赖；不能脱离 Spring 容器运行，所以现在已经不推荐使用。

接下来是 Spring 实现依赖注入的完整流程。整个过程是 IOC 容器工作流程的核心环节，分为四个关键步骤：第一是依赖解析，容器在解析 BeanDefinition 时，会识别出 Bean 声明的所有依赖关系，比如通过 `@Autowired` 注解标记的字段、构造器参数或者 setter 方法；第二是依赖查找，容器根据依赖的类型和名称，从 BeanFactory 中查找对应的依赖 Bean 实例；第三是依赖注入，容器通过反射机制，将查找到的依赖对象注入到目标 Bean 中：构造器注入是反射调用构造方法传入参数，setter 注入是反射调用 setter 方法，字段注入是反射直接设置字段值；第四是循环依赖处理，如果出现单例 Bean 之间的循环依赖，Spring 会通过三级缓存机制提前暴露半成品对象，**解决循环注入问题**。

> **@Autowired vs @Resource（高频辨析）**

> - `@Autowired`（Spring 注解）：**默认按类型**；同类型多个候选时，**按字段名 / 参数名收窄**，还不行就抛 `NoUniqueBeanDefinitionException`（可用 `@Primary` 或 `@Qualifier` 解决）；
> - `@Resource`（JSR-250 标准，JDK 自带）：**默认按名称**，找不到再按类型。
>
> 一句话：**Autowired 先类型后名字，Resource 先名字后类型。**

然后是关键设计细节。第一是**自动装配模式**，Spring 提供了三种自动装配方式：`byName` 根据属性名称匹配 Bean，`byType` 根据属性类型匹配 Bean，`constructor` 根据构造器参数类型匹配 Bean，默认是 `byType`。第二是常用注解区别：`@Autowired` 是 Spring 提供的注解，默认按类型匹配，找不到再按名称匹配；`@Resource` 是 JSR-250 标准注解，默认按名称匹配，找不到再按类型匹配；`@Qualifier` 配合 `@Autowired` 使用，当有多个同类型 Bean 时，指定具体的 Bean 名称。第三是依赖注入的时机，单例 Bean 在容器启动时完成注入，原型 Bean 在每次获取时才进行注入。

Q4. spring是如何解决循环依赖的?

关于 Spring 如何解决循环依赖，我从核心本质、什么是循环依赖、三级缓存核心机制、完整解决流程、无法解决的场景、常见误区和实际开发应用六个方面来回答：

首先，Spring 解决循环依赖的核心本质是通过三级缓存机制**提前暴露半成品**对象，让循环依赖的 Bean 能够互相引用。**循环依赖**指的是两个或多个 Bean 之间相互持有对方的引用，形成闭环，比如 A 依赖 B，B 又依赖 A。如果没有特殊处理，容器在创建 A 时需要 B，创建 B 时又需要 A，会导致无限递归创建对象，最终抛出栈溢出异常。

接下来是循环依赖的分类和可解决性。Spring 只能解决单例 Bean 的 setter 方法循环依赖，这也是最常见的循环依赖场景。有三种情况 Spring 无法解决：第一是构造器循环依赖，因为构造器注入时对象还没创建，无法提前暴露；第二是多例 Bean 的循环依赖，因为多例 Bean 每次获取都创建新实例，没有缓存机制；第三是使用 @Async、@Transactional 等 AOP 注解导致的特殊循环依赖，可能会出现代理对象无法提前暴露的问题。

然后是 Spring 的三级缓存核心机制，这是解决循环依赖的关键。Spring 在 IOC 容器中维护了三个 Map 结构的缓存：第一级缓存是singletonObjects，也叫单例池，存放已经完全初始化完成的 Bean 实例；第二级缓存是earlySingletonObjects，存放已经实例化但还未完成属性注入和初始化的半成品 Bean；第三级缓存是singletonFactories，存放 Bean 的对象工厂 ObjectFactory，用于提前暴露 Bean 的引用。

这里我想重点讲一下为什么需要三级缓存，而不是两级。很多人以为三级缓存是为了解决循环依赖，其实核心是为了处理 AOP 代理对象。如果没有 AOP，两级缓存就足够了；但如果 Bean 需要被代理，三级缓存中的 ObjectFactory 会在需要时生成代理对象，而不是直接暴露原始对象，这样就能保证注入的是代理对象而不是原始对象。

接下来是**完整的循环依赖解决流程**，我以 A 和 B 互相依赖为例来说明：

> **第一步**，容器开始创建 A，先实例化 A（调用构造方法），此时 A 还没有完成属性注入和初始化，是一个半成品；

第二步，将 A 的对象工厂放入三级缓存 singletonFactories 中，提前暴露 A 的引用；

第三步，开始给 A 注入属性 B，发现 B 还没有创建，于是转而去创建 B；

第四步，实例化 B，同样将 B 的对象工厂放入三级缓存；

第五步，给 B 注入属性 A，此时从三级缓存中获取 A 的对象工厂，生成 A 的引用（如果需要代理则生成代理对象），将 A 放入二级缓存 earlySingletonObjects 中，然后将 A 注入到 B 中；

第六步，B 完成所有属性注入和初始化，成为完整的 Bean，放入一级缓存 singletonObjects 中；

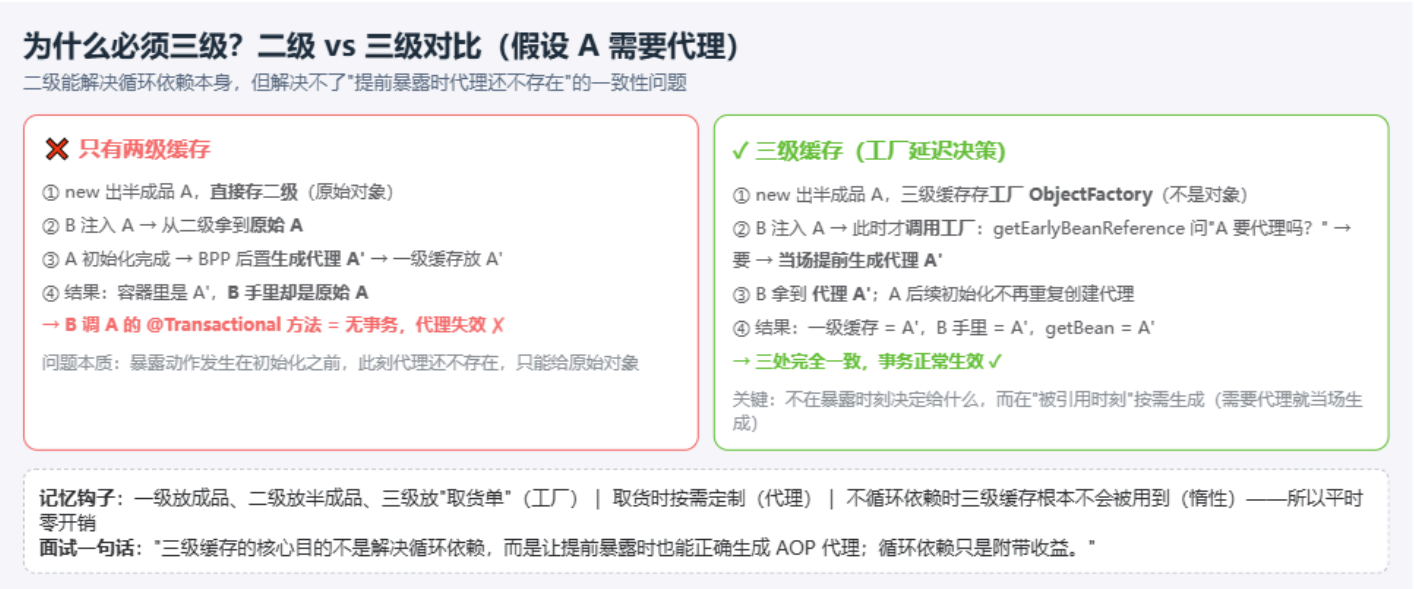
第七步，回到 A 的创建过程，将已经创建好的 B 注入到 A 中；

第八步，A 完成所有初始化，从二级缓存升级到一级缓存，循环依赖解决完成。

然后是**无法解决的场景及原因**。第一，构造器循环依赖无法解决，因为构造器注入时对象还没有实例化，无法提前暴露到缓存中；第二，多例 Bean 循环依赖无法解决，因为多例 Bean 不会被缓存，每次获取都创建新实例，会导致无限递归；第三，@Async 注解导致的循环依赖无法解决，因为 @Async 会在 Bean 初始化完成后通过 BeanPostProcessor 生成代理对象，而三级缓存提前暴露的是原始对象，注入后会导致代理对象不生效。

这里我想重点澄清两个常见误区：第一，不是所有循环依赖 Spring 都能解决，只有单例 Bean 的 setter 循环依赖可以解决；第二，三级缓存的主要目的不是为了解决循环依赖，而是为了统一处理 AOP 代理对象，解决循环依赖只是它的一个附加功能。

项目锚点：券享购 Service 层 OrderService 依赖 CouponService、CouponService 反查 OrderService 做核销回写——Spring 三级缓存自动解决；若用构造器注入会直接报错，逼着重构拆职责。@Transactional 的循环依赖能被三级缓存解决（TransactionAttributeSourceAdvisor 实现了 getEarlyBeanReference），但 @Async 不行——这是真实踩坑点。简历点名 Spring IOC/AOP 深挖。



> **收尾**：把这个故事串成背诵链
>
> 1. **问题**：A↔B 互相依赖，谁都无法先完成 → 递归栈溢出；
> 2. **钥匙**：提前暴露半成品；

3. **一级不行**：成品池不能放坏对象（语义破坏）；
4. **二级解决了循环依赖、但代理会失效**：暴露发生在初始化前，代理还不存在，B 拿到原始对象 → 事务没了；
5. **三级是答案**：存工厂而非对象，被引用那一刻调用 `getEarlyBeanReference` 按需生成代理 → 所有人都拿到同一个代理；
6. **三个死结**：构造器（没有半成品可暴露）、多例（没有缓存）、@Async（生成器没实现 getEarlyBeanReference）。

Q5. MVC分层介绍一下（含MVC的完整工作流程）

关于 MVC 分层架构，我从核心本质、三层各自职责、完整工作流程、核心优势、常见误区和实际开发应用六个方面来回答：

首先，MVC 的核心本质是一种经典的软件架构设计模式，全称是 Model-View-Controller，它的核心思想是关注点分离，将一个应用程序拆分为数据模型、用户界面和控制逻辑三个独立的部分，让每一层只负责自己的职责，互不干扰。这样可以极大地降低代码的耦合度，提高代码的可维护性、可扩展性和可测试性，是几乎所有 Web 应用开发的标准架构。

接下来是三层各自的核心职责，这是 MVC 的基础。第一层是Model 模型层，负责处理应用程序的数据和业务逻辑，是整个应用的核心。它不关心数据如何展示，也不关心用户如何交互，只负责接收 Controller 的请求，处理业务逻辑，与数据库进行交互，然后返回处理结果。Model 层通常又会细分为 Service 业务层和 Repository 数据访问层，Service 层处理业务逻辑，Repository 层负责与数据库交互。第二层是View 视图层，负责用户界面的展示，将 Model 层返回的数据以合适的形式呈现给用户。它只负责展示数据，不处理任何业务逻辑，也不直接访问数据库。在前后端分离的架构中，View 层就是前端页面，比如 Vue、React 写的页面；在传统的服务端渲染中，就是 JSP、Thymeleaf 等模板引擎。第三层是Controller 控制层，是连接 Model 和 View 的桥梁，负责接收用户的请求，进行参数校验，然后调用对应的 Model 层方法处理业务逻辑，最后将处理结果返回给 View 层进行展示。它只负责调度，不处理任何业务逻辑。

然后是 **MVC 的完整工作流程**。当用户发送一个 HTTP 请求时，首先请求会到达 Controller；Controller 接收请求后，进行参数解析和校验，然后调用对应的 Service 层方法；**Service 层执行业务逻辑**，调用 Repository 层方法访问数据库；**Repository 层执行 SQL 语句**，从数据库中获取数据，返回给 Service 层；Service 层处理完业务逻辑后，将结果返回给 Controller；Controller 将结果封装成 ModelAndView 对象，传递给 View 层；View 层根据 Model 中的数据渲染页面，最后将渲染好的页面返回给用户。在前后端分离的架构中，Controller 会直接返回 JSON 数据给前端，由前端进行渲染。

> **完整流程**：HTTP 请求 → Controller 接收 + 参数校验 → Service 业务逻辑 → Repository 访问数据库 → 结果逐层返回 → Controller 封装（前后端分离直接返回 JSON，由前端渲染；传统模式返回 ModelAndView 交给 View 渲染）→ 返回用户。

接下来是 MVC 的核心优势。第一，关注点分离，每一层只负责自己的职责，代码结构清晰，易于理解和维护；第二，代码复用性高，Model 层的业务逻辑可以被多个 Controller 复用，View 层的页面组件

也可以被多个页面复用；第三，便于测试，每一层都可以独立进行单元测试，不需要依赖其他层；第四，支持并行开发，不同的开发人员可以同时开发不同的层，比如前端开发人员写 View 层，后端开发人员写 Model 和 Controller 层，提高开发效率；第五，易于扩展，当需要修改功能时，只需要修改对应的层即可，不会影响其他层的代码。

这里我想重点澄清常见误区。第一，很多人以为 MVC 是 Spring MVC 独有的，其实不是，MVC 是一种通用的架构设计模式，~~早在 Spring 出现之前就已经存在了，Spring MVC 只是 MVC 模式的一种具体实现~~

Q6. 了解Spring Cloud Alibaba吗，说一下他和SpringBoot的区别

关于 Spring Cloud Alibaba 和 Spring Boot 的区别，我从核心本质、各自定位、核心功能对比、技术生态关系、版本依赖规则、常见误区和实际开发应用七个方面来回答：

首先，两者的核心本质和定位完全不同，不是竞争关系而是依赖关系。Spring Boot 是快速开发单个 Java 应用的基础框架，核心目标是 "约定大于配置"，简化单个应用的开发、配置和部署；而 Spring Cloud Alibaba 是基于 Spring Boot 构建的微服务架构一站式解决方案，核心目标是解决分布式微服务系统中的服务治理、配置管理、流量控制、分布式事务等共性问题。简单来说：Spring Boot 造轮子，Spring Cloud Alibaba 用轮子组装成一辆完整的微服务汽车。

然后是核心功能对比，这是两者最直观的区别。Spring Boot 的核心功能聚焦于单个应用：第一，自动配置，根据类路径中的 jar 包自动配置 Bean；第二，起步依赖，将常用依赖打包成 starter，简化 Maven 配置；第三，嵌入式服务器，内置 Tomcat、Jetty 等服务器，无需单独部署；第四，Actuator 应用监控，提供健康检查、指标监控等端点；第五，外部化配置，支持从配置文件、环境变量、命令行参数加载配置。而 Spring Cloud Alibaba 的核心功能聚焦于分布式系统：第一，Nacos，同时提供服务注册发现和分布式配置中心；第二，Sentinel，提供流量控制、熔断降级、系统负载保护；第三，Seata，提供高性能的分布式事务解决方案；第四，RocketMQ，提供消息驱动和异步通信能力；第五，Dubbo，提供高性能的 RPC 远程调用；第六，Alibaba Cloud，无缝集成阿里云的 OSS、SMS、Redis 等云服务。

然后是版本依赖规则，这是面试必考点也是开发中最容易踩坑的地方。Spring Cloud Alibaba 有严格的版本对应关系，它同时依赖 Spring Boot 和 Spring Cloud 的特定版本。比如 Spring Cloud Alibaba 2021.0.5.0 版本对应 Spring Boot 2.6.13 和 Spring Cloud 2021.0.5 版本；Spring Cloud Alibaba 2022.0.0.0 版本对应 Spring Boot 3.0.2 和 Spring Cloud 2022.0.0 版本。如果版本不匹配，会出现各种奇怪的兼容性问题，甚至导致应用无法启动。

这里我想重点澄清两个非常常见的误区：第一，很多人以为 Spring Cloud Alibaba 是 Spring Boot 的替代品，其实完全不是，没有 Spring Boot 就没有 Spring Cloud Alibaba，每个微服务应用本身就是一个 Spring Boot 应用；第二，很多人以为 Spring Cloud Alibaba 只能使用阿里巴巴的组件，其实不是，它完全兼容 Spring Cloud 的标准，你可以同时使用 Nacos 做服务注册，用 OpenFeign 做远程调用，用 Gateway 做网关。【项目中就是如此】

最后是实际开发中的应用。在微服务项目中，我会这样使用两者：首先，每个微服务模块都是一个独立的 Spring Boot 应用，使用 Spring Boot 的自动配置和起步依赖快速搭建；然后，在父项目中统一

引入 Spring Cloud Alibaba 的依赖管理，统一管理所有组件的版本；接着，使用 Nacos 作为服务注册中心，让所有微服务可以互相发现和调用；使用 Nacos 作为配置中心，统一管理所有微服务的配置；使用 Sentinel 对核心接口进行流量控制和熔断降级，保护系统稳定性；使用 Seata 解决分布式事务问题，保证数据一致性；使用 Gateway 作为微服务网关，统一处理路由、认证、限流等横切关注点。同时，我会严格遵循官方的版本对应关系，避免出现兼容性问题。

> 在项目中，每个微服务都是一个 Spring Boot 应用，它们通过 Spring Cloud Alibaba 的 Nacos 做服务注册和配置中心，通过 OpenFeign 做远程调用，通过 Sentinel 做限流熔断，通过 RocketMQ 做异步解耦，通过 Spring Cloud Gateway 做统一入口——这才是完整的微服务架构体系。

> **常用微服务组件**（按功能分类）：
>
> 1. 服务治理：Nacos——注册发现 + 健康检查，支持 AP/CP 切换。
> 2. 配置管理：Nacos 配置中心——热更新、命名空间/分组隔离环境、版本回滚。
> 3. 流量控制/熔断：Sentinel——核心接口限流，第三方依赖熔断降级防雪崩。
> 4. 远程调用：OpenFeign——声明式 HTTP 客户端，可集成 Sentinel 熔断。
> 5. 网关：Spring Cloud Gateway——统一路由、认证、日志、跨域、全局限流。
> 6. 消息队列：RocketMQ——异步解耦、削峰填谷（详见第 6 章）。
> 7. 分布式事务：Seata——AT 模式基于两阶段提交，对业务无侵入。
> 8. 链路追踪：SkyWalking——调用链拓扑、快速定位故障服务。

Q7. 用过哪些微服务组件？

关于我使用过的微服务组件，我从核心本质、核心组件分类及使用经验、组件协同工作流程、常见问题解决方案、最佳实践和实际项目应用六个方面来回答：

首先，微服务组件的核心本质是解决分布式系统的共性问题，将服务治理、配置管理、流量控制等通用能力抽离成独立组件，让开发者专注于业务逻辑开发。我主要使用的是Spring Cloud Alibaba生态的组件，这也是目前国内微服务开发的主流选择，同时也会配合使用部分 Spring Cloud 原生组件和第三方中间件。

接下来是我在项目中实际使用过的核心组件，按照功能分为八大类：

- 第一类是**服务治理组件**，使用的是Nacos，它同时提供服务注册发现和服务健康检查功能。在项目中，所有微服务都会注册到 Nacos 注册中心，服务之间通过服务名进行调用，Nacos 会自动维护服务实例的上下线状态，实现了服务的动态发现和负载均衡。相比传统的 Eureka，Nacos 支持 AP 和 CP 两种模式切换，性能更好，功能也更完善。

- 第二类是**配置管理组件**，同样使用Nacos作为分布式配置中心。将所有微服务的配置统一管理在Nacos 中，支持配置的热更新，不需要重启服务就能生效。我们还使用了命名空间和分组功能，区分开发、测试、生产环境的配置，同时支持配置的版本控制和回滚，极大地提升了配置管理的效率。
- 第三类是流量控制与熔断降级组件，使用的是Sentinel。在项目中，我们对所有核心接口都配置了流量控制规则，防止突发流量打垮服务；同时对依赖的第三方服务和慢接口配置了熔断降级规则，当服务出现异常时，快速失败并返回降级结果，避免服务雪崩。Sentinel 提供了实时监控控制台，可以直观地看到各个接口的流量和异常情况，非常方便运维。
- 第四类是**远程调用组件**，使用的是OpenFeign，它是 Spring Cloud 原生的声明式 HTTP 客户端。通过在接口上添加 `@FeignClient` 注解，就可以像调用本地方法一样调用远程服务，非常简洁。我们还集成了 Sentinel 和 OpenFeign，实现了远程调用的熔断降级功能。
- 第五类是**API 网关组件**，使用的是Spring Cloud Gateway。所有客户端的请求都先经过网关，由网关统一路由到对应的微服务。在网关层我们实现了统一的认证授权、请求日志记录、跨域处理、全局限流等功能，避免了在每个微服务中重复实现这些逻辑。
- 第六类是**消息队列组件**，使用的是RocketMQ，它是阿里巴巴开源的高性能消息中间件。在项目中，我们用它来实现异步解耦、流量削峰和分布式事务。比如订单创建成功后，发送消息通知库存服务扣减库存，通知用户服务发送短信通知，这样既提高了系统的吞吐量，又实现了服务之间的解耦。
- 第七类是**分布式事务组件**，使用的是Seata，它提供了 AT、TCC、SAGA 等多种分布式事务模式。我们主要使用 AT 模式，它是基于两阶段提交的，对业务代码几乎无侵入，能够保证跨微服务的数据一致性。在订单支付、库存扣减等需要强一致性的场景中，我们都使用了 Seata 来解决分布式事务问题。
- ~~第八类是链路追踪组件~~，使用的是SkyWalking。它能够自动采集分布式系统中的调用链路数据，生成可视化的调用拓扑图和链路详情。当系统出现问题时，我们可以通过 SkyWalking 快速定位到哪个服务、哪个接口出现了异常，以及异常的具体原因，大大提高了问题排查的效率。

然后是这些组件的**协同工作流程**。当用户发送一个请求时，首先请求到达 Spring Cloud Gateway 网关；网关进行认证授权和路由匹配后，将请求转发到对应的微服务；微服务通过 OpenFeign 调用其他依赖的服务，所有的服务调用都通过 Nacos 进行服务发现；Sentinel 对所有接口进行流量控制和熔断降级；如果涉及到跨服务的事务，使用 Seata 保证数据一致性；所有的调用链路数据都被 SkyWalking 采集和监控；服务之间的异步通信通过 RocketMQ 实现。

这里我想重点讲**两个实际项目中遇到的问题和解决方案**：第一个是服务雪崩问题，我们曾经遇到过一个下游服务响应变慢，导致上游服务线程池被占满，整个系统崩溃的情况。后来我们通过 Sentinel 对下游服务的调用接口配置了熔断降级规则，当异常率超过阈值时，自动熔断，快速返回降级结果，保护了上游服务的稳定性。第二个是配置不一致问题，之前我们的配置是分散在各个微服务的配置文件中，修改配置需要重启服务，非常麻烦。后来我们将所有配置迁移到 Nacos 配置中心，实现了配置的统一管理和热更新，大大提高了运维效率。

常见误区部分，我想澄清两个点：第一，微服务不是拆得越细越好，过度拆分反而会增加系统的复杂度和运维成本，应该根据业务边界进行合理拆分；第二，组件不是越多越好，应该根据业务需求选择合适的组件，避免引入不必要的复杂度。

Q8. Spring Bean 的生命周期说一下？

面试官您好，关于 Spring Bean 的生命周期，我从核心定义、完整流程、扩展点应用和实际项目实践四个方面来回答：

首先，Bean 的生命周期是指 Spring 容器中一个 Bean 从创建、初始化到销毁的完整过程。核心本质是容器对 Bean 全生命周期的统一托管，理解它就能理解 Spring 的各种扩展机制是怎么插入进去的。

完整的生命周期分为八个阶段：第一，实例化，容器通过反射调用构造器创建 Bean 对象；第二，属性填充，容器完成依赖注入，把 @Autowired 等注解声明的依赖注入进来；第三，Aware 接口回调，如果 Bean 实现了 BeanNameAware、BeanFactoryAware、ApplicationContextAware，容器会回调对应方法注入容器信息；第四，BeanPostProcessor 前置处理，执行 postProcessBeforeInitialization 方法；第五，初始化，按顺序执行 @PostConstruct 注解方法、InitializingBean 接口的 afterPropertiesSet 方法、init-method 配置的初始化方法；第六，BeanPostProcessor 后置处理，执行 postProcessAfterInitialization 方法，这里最关键——AOP 动态代理就是在这个阶段生成的，所以代理对象在初始化完成后才生效；第七，Bean 就绪，被业务代码正常使用；第八，销毁，容器关闭时按顺序执行 @PreDestroy 注解方法、DisposableBean 的 destroy 方法、destroy-method 配置的销毁方法。

这里我想重点讲一个高频面试点：Spring 为什么把扩展点设计成这么多层？核心原因是分层解耦——实例化和属性填充是容器的基础职责，Aware 是让 Bean 感知容器的窗口，BeanPostProcessor 是框架级扩展点（AOP、@Async 都靠它织入），而 @PostConstruct / InitializingBean 是业务级初始化入口。面试官如果追问，可以这样回答：BeanPostProcessor 作用于所有 Bean，是框架级钩子；InitializingBean 和 init-method 只作用于当前 Bean，是业务级钩子。

在实际项目中，我用 @PostConstruct 做初始化场景比较多，比如缓存预热、线程池初始化、启动时加载配置到内存；用 ApplicationRunner 或 CommandLineRunner 做应用启动后的整体任务（如本项目责任链注册表就是在 run() 中完成的）。同时我会注意：如果 Bean 需要代理，AOP 代理在初始化之后生成，所以在 @PostConstruct 里拿到的 this 是原始对象，需要代理能力时要通过 ApplicationContext.getBean 获取。

Q9. Spring 事务的传播行为有哪些？

面试官您好，关于 Spring 事务的传播行为，我从核心定义、七种传播行为、重点对比和项目实践四个方面来回答：

首先，事务传播行为解决的核心问题是：当一个事务方法调用另一个事务方法时，这两个方法的事务该如何组织。Spring 定义了七种传播行为，其中最常用的是三种。

第一种是 REQUIRED，默认传播行为。如果当前存在事务，就加入当前事务；如果当前没有事务，就新建一个事务。绝大多数业务方法都用它，保证一个调用链上的数据库操作在同一个事务里，要么全部成功要么全部回滚。第二种是 REQUIRES_NEW，无论当前有没有事务，都挂起当前事务，新建一个独立的新事务。新事务提交或回滚不影响外层事务。第三种是 NESTED，嵌套事务，如果当前存在事务，创建一个保存点，内层回滚只回滚到保存点，外层事务可以继续提交；如果当前没有事务，行为和 REQUIRED 一样。其余四种：SUPPORTS 有事务就加入，没有就以非事务方式执行；NOT_SUPPORTED 以非事务方式执行，挂起当前事务；MANDATORY 强制要求当前存在事务，否则抛异常；NEVER 强制要求当前没有事务，否则抛异常。

> **REQUIRED 复用连接同生共死；REQUIRES_NEW 换连接兄弟分家；NESTED 不换连接只打保存点。**

这里我想重点对比两个最容易混淆的传播行为：REQUIRES_NEW 和 NESTED。REQUIRES_NEW 是完全独立的新事务，内层回滚不影响外层，外层回滚也不影响已经提交的内层事务；NESTED 是嵌套事务，内层回滚只回滚到保存点，外层可以捕获异常后继续，但如果外层最终回滚，内层也会跟着回滚。简单说，REQUIRES_NEW 是兄弟关系，NESTED 是父子关系。

在实际项目中，默认业务方法用 REQUIRED；写操作日志、埋点、审计这类场景用 REQUIRES_NEW——因为日志写失败不应该回滚主业务事务，否则主业务成功但日志失败导致整体回滚，会造成严重问题；对于大批量数据处理中需要部分成功的场景，会考虑 NESTED。另外要注意一个经典坑：同一个类内部方法调用，@Transactional 注解不会生效（自调用不走代理），必须通过注入自身代理或者拆到不同 Bean 中。



Q10. Spring Boot 自动配置原理说一下?

> @SpringBootApplicatio{三合一注解}， 包含以下三个注解

注解	本质	职责
@SpringBootConfiguration	就是 @Configuration	声明这是一个配置类（容器入口）
@ComponentScan	组件扫描	扫描**启动类所在包及子包**的 @Component/ @Service/@Controller/@Configuration
@EnableAutoConfiguration	核心	**开启自动配置**（真正的 "魔法" 在这里）

@EnableAutoConfiguration 五步走

```
① @EnableAutoConfiguration 的本质 = @Import(AutoConfigurationImportSelector.class)
        ↓
② AutoConfigurationImportSelector 是 ImportSelector 的扩展（DeferredImportSelector）
   它做的事情：读取 jar 包里的配置文件，拿到"自动配置类的名单"
        ↓
③ 读取的文件（注意版本差异）：
   Boot 2.7 之前：META-INF/spring.factories（key = EnableAutoConfiguration）
   Boot 2.7 起：  META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports
        ↓
④ 名单里有成百上千个自动配置类（RedisAutoConfiguration、DataSourceAutoConfiguration、
   JacksonAutoConfiguration……），一行一个全限定名
        ↓
⑤ 但【不会全部生效】——交给条件注解逐类判断 @Conditional 家族
```

> 为什么用 DeferredImportSelector（延迟导入）？
>
> 让自动配置类**延迟到用户自定义的 @Configuration 全部处理完之后**再处理 —— 保证 "用户自定义优先、自动配置兜底" 的顺序。

名单有几百个自动配置类，为什么不会全部加载？**每个自动配置类上都挂着条件注解，条件不满足就跳过**。这就是 "按需加载"。

条件注解	作用	例子
@ConditionalOnClass	**classpath 里有这个类**才生效	没引 Redis 依赖 → RedisAutoConfiguration 直接跳过
@ConditionalOnMissingBean	**容器里没有这个 Bean**才生效	用户已自定义 RedisTemplate → 自动配置不重复创建
@ConditionalOnProperty	配置项满足才生效	spring.redis.* 没配置就不绑
@ConditionalOnWebApplication	是 Web 环境才生效	非 Web 项目不加载 MVC 配置

面试官您好，关于 Spring Boot 的自动配置原理，我从核心定义、启动注解拆解、自动配置加载机制和实际应用四个方面来回答：

首先，自动配置是 Spring Boot "约定大于配置"思想的核心实现，它的作用是：根据 classpath 下的依赖 jar 包，自动创建并配置好对应的 Bean，让开发者开箱即用，不用再写一堆 XML 或 @Configuration 配置类。

接下来是核心机制。入口是启动类上的 @SpringBootApplication 注解，它是一个组合注解，由三个注解组成：@SpringBootConfiguration 本质上是一个 @Configuration，标记这是一个配置类；@ComponentScan 开启组件扫描，扫描启动类所在包及子包；最关键的是 @EnableAutoConfiguration，它通过 @Import 导入了一个 AutoConfigurationImportSelector 类，这个类会读取所有 jar 包中 META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports 文件（Spring Boot 2.7 之前是 spring.factories），拿到所有自动配置类的全限定名，然后逐一加载。

但是，自动配置类不会全部生效，它通过条件注解实现按需加载：@ConditionalOnClass 判断 classpath 下有没有对应依赖类，比如没有引入 Redis 的 jar 包，RedisAutoConfiguration 就不会生效；@ConditionalOnMissingBean 判断容器中是否已有用户自定义的 Bean，如果用户自己定义了，就以后者的配置为准；还有 @ConditionalOnProperty 判断配置项是否存在。同时自动配置类配合 @ConfigurationProperties 实现配置绑定，比如 spring.datasource.* 的配置会自动绑定到 DataSourceProperties 中。最后，每个自动配置类内部通常会配合 @EnableConfigurationProperties 和 @ConditionalOnMissingBean 给出默认 Bean，同时允许用户通过自定义 Bean 覆盖默认行为。

最后是实际应用。在工作中，我会利用这个机制排查问题：当某个自动配置没生效时，先检查 classpath 依赖是否存在、条件注解是否满足、配置项是否拼写正确；通过 spring-boot-autoconfigure 源码可以看到每个自动配置类上的 @ConditionalOnXxx 条件，快速定位。自定义 starter 时，就是按同样的约定创建 AutoConfiguration 类并注册到 AutoConfiguration.imports 文件中，再用条件注解控制生效范围。同时记住一个面试高频点：自动配置类的加载优先级低于用户自定义的 @Configuration 和 @Component，所以用户总能通过自定义 Bean 覆盖默认配置。



Java 基础语法高频问题

String

Q1. String、StringBuilder、StringBuffer 的区别?

面试官您好，关于 String 三兄弟的区别，我从核心本质、可变性、线程安全和适用场景四个方面来回答：

首先，三者的核心区别是可变性：String 是不可变类，底层用 final char 数组（JDK9 之后是 byte 数组）存储，一旦创建内容不可修改；StringBuilder 和 StringBuffer 是可变类，底层同样是数组但可以动态扩容修改内容。

接下来是关键差异。第一，为什么 String 要设计成不可变？一是安全，字符串常量池共享、类加载、网络参数等场景都依赖不可变；二是线程安全，不可变对象天然线程安全；三是缓存，哈希值可以缓存，HashMap 用 String 做 key 性能好。第二，StringBuilder 和 StringBuffer 的区别是线程安全：StringBuffer 的方法都加了 synchronized，线程安全但性能差；StringBuilder 不加锁，线程不安全但性能最高，是单线程环境下的唯一推荐。

第三，拼接性能。字符串用 + 拼接时，编译器和 JVM 会优化，简单拼接常量会在编译期直接合并，变量拼接会优化成 StringBuilder.append。但在循环里拼接，每次循环都会 new 一个 StringBuilder，

性能很差，所以循环拼接必须显式使用 StringBuilder。第四，扩容机制，StringBuilder 和 ArrayList 类似，默认容量 16，扩容时新容量 = 旧容量 * 2 + 2。

最后是适用场景总结：字符串内容不变的场景用 String；单线程下频繁拼接、修改用 StringBuilder；多线程环境需要线程安全且要拼接时用 StringBuffer（实际项目中这种场景很少，一般直接加锁或用别的方案）。

Q2. String 的 intern() 方法有什么用？

面试官您好，关于 intern() 方法，我从核心作用、底层原理和实际应用三个方面来回答：

首先，intern() 的核心作用是返回字符串在字符串常量池中的引用。JDK7 之前常量池在永久代，JDK7 之后移到堆中。调用 intern() 时，如果常量池中已有内容相同的字符串，直接返回池中对象的引用；如果没有，JDK7 及以后会把当前字符串对象的引用放入常量池（不会复制对象），再返回该引用。

这里有一个经典面试题：String s1 = new String("a") + new String("b") 创建了几个对象？答案是：s1 本身一个对象，"a" 和 "b" 各一个常量池对象，以及 new 出来的两个 String 对象，如果后续调用 s1.intern() 还会把 "ab" 的引用放入常量池。面试官常考的是 new String("abc") 创建几个对象——答案是 2 个：一个常量池对象，一个堆对象。

最后是实际应用。intern() 可以用于字符串去重，大量相同字符串时节省内存（如 JVM 的 StringDeduplication 就是类似思想），但 JDK8 之后字符串常量池也受堆大小限制，滥用 intern() 反而可能因为常量池过大导致性能问题，生产环境要谨慎。

异常

Q3. Java 的异常体系说一下？

面试官您好，关于 Java 异常体系，我从核心结构、受检异常与非受检异常的区别、异常处理原则和项目实践四个方面来回答：

首先，Java 所有异常和错误的根类是 Throwable，它分为两大类：Error 和 Exception。Error 是 JVM 层面的严重错误，比如 OutOfMemoryError、StackOverflowError、NoClassDefFoundError，程序无法处理也不应该捕获。Exception 是程序可以处理的异常，又分为 RuntimeException 运行时异常和受检异常 Checked Exception。

受检异常必须显式处理（捕获或抛出），比如 IOException、SQLException、FileNotFoundException，编译器强制检查；非受检异常（RuntimeException 及其子类）编译器不强制处理，比如 NullPointerException、IllegalArgumentException、ClassCastException、IndexOutOfBoundsException。

然后是异常处理的几个关键点：第一，try-catch-finally 中 finally 一定会执行（除非 System.exit），用于释放资源；第二，JDK7 引入 try-with-resources，自动关闭实现 AutoCloseable 的资源，比 finally 更简洁，是生产环境的标准写法；第三，catch 多个异常时，子类异常必须写在父类异常前面，

否则编译报错；第四，throws 和 throw 的区别：throws 声明方法可能抛出的异常，throw 是实际抛出异常对象。

最后是项目实践中的原则：第一，能用 try-with-resources 就不用 finally 手动关闭；第二，catch 之后要么记录日志要么抛出有意义的异常，禁止吞异常（catch 了什么都不做）；第三，业务异常建议自定义异常类（继承 RuntimeException，配合全局异常处理器 @RestControllerAdvice 统一返回错误码），我在项目中用 RRException 统一处理，Redis 故障时抛出并返回友好提示；第四，不在循环内捕获异常，避免性能损耗和日志刷屏。

泛型

Q4. 泛型是什么？泛型擦除是什么？

面试官您好，关于泛型，我从核心作用、泛型擦除原理、通配符和实际应用四个方面来回答：

首先，泛型的核心作用是编译期类型安全，让集合等容器在编译阶段就约束元素类型，避免运行期强转的 ClassCastException。Java 的泛型是伪泛型，JDK5 引入，底层通过类型擦除实现。

泛型擦除的核心原理是：编译时泛型类型信息会被擦除，泛型类型参数在字节码中被替换为它的边界类型（无边界就替换为 Object），并在必要的位置插入强制类型转换。比如 List<String> 和 List<Integer> 在字节码层面都是 List，运行时 getClass() 相同；泛型方法擦除后参数变成 Object，调用处插入 cast 指令。

然后是泛型的几个关键点：第一，泛型不支持基本类型，List<int> 不合法，要用包装类；第二，泛型不支持静态上下文，static 字段和方法不能使用类泛型参数；第三，泛型不能直接 new T() 创建实例，因为类型被擦除了；第四，泛型数组不能直接创建，new T[] 不合法，因为数组在运行时会检查类型。第五，通配符：<? extends T> 表示 T 或 T 的子类，只读不写（上界通配符）；<? super T> 表示 T 或 T 的父类，只写不读（下界通配符）。这就是 PECS 原则：Producer Extends，Consumer Super——只往外读数据用 extends，只往里写数据用 super。

最后是实际应用。项目中泛型主要用在：集合类型约束、通用返回结果类 Result<T>、通用分页对象 PageResult<T>、通用工具类的类型安全转换。面试高频题：如何绕过泛型擦除向 List<Integer> 中放字符串？通过反射调用 add(Object) 方法可以做到，这证明了擦除的存在。

反射

Q5. 反射是什么？有哪些应用场景？

面试官您好，关于反射，我从核心定义、常用 API、底层原理、应用场景和注意事项五个方面来回答：

首先，反射是 Java 在运行期获取类的完整结构信息（类名、方法、字段、注解、父类、接口）并动态操作对象的能力。核心类有三个：Class 代表类本身，Method 代表方法，Field 代表字段。获取 Class 对象有三种方式：类名.class、对象.getClass()、Class.forName("全限定名")，其中 forName 最常用，因为它可以在运行期按字符串动态加载类。

反射的核心应用场景：第一，Spring IOC 的依赖注入就是通过反射读取类信息并实例化对象、注入依赖；第二，JDBC 的 Class.forName 加载驱动；第三，动态代理 JDK 实现基于反射生成代理类；第四，框架层面的注解处理，比如 Spring 扫描 @Component 注解并通过反射创建 Bean；第五，序列化框架 Jackson、Fastjson 通过反射读写对象字段。

然后是反射的底层原理和性能：反射调用比直接调用慢，主要开销在方法查找、安全检查、参数装箱拆箱，JDK 内部有反射调用的优化（如 MethodHandle、膨胀阈值），但业务代码中高频路径应避免使用反射。同时反射可以访问私有成员（setAccessible(true)），会破坏封装，也有安全风险，禁止在不可信输入上直接使用反射。

最后是注意事项：第一，反射调用性能敏感场景要缓存 Method/Field 对象，不要每次都 getMethod；第二，获取泛型信息要用 Type 而不是 Class；第三，反射抛出的 InvocationTargetException 包装了真实异常，要用 getCause() 取原始异常。Spring 中我用的 @Autowired 字段注入底层就是反射赋值，构造器注入底层是反射调构造方法。

IO / NIO

Q6. BIO、NIO、AIO 的区别?

面试官您好，关于 IO 模型，我从核心定义、三者区别、NIO 核心组件和实际应用四个方面来回答：

首先，BIO 是阻塞 IO，每个连接对应一个线程，线程在 read/write 时阻塞等待数据，连接多了线程数爆炸，不适合高并发；NIO 是非阻塞 IO，基于 Selector 多路复用器，一个线程可以管理多个连接，连接有事件才处理，适合连接多但活跃度低的场景；AIO 是异步 IO，读写操作提交后立即返回，完成时回调通知，Java 中 AIO 实际应用很少，Linux 下的异步 IO 支持也不完善，一般不提。

NIO 的三个核心组件：Channel 通道（双向，类似流但可读可写）、Buffer 缓冲区（读写都经过 Buffer，flip 切换读写模式）、Selector 选择器（注册多个 Channel，监听 OP_ACCEPT、OP_READ、OP_WRITE 事件）。NIO 的经典应用就是 Netty，而 Redis 的单线程模型也是基于 IO 多路复用（epoll），这和 NIO 是同一个思想。

这里我想重点说明 NIO 和 BIO 的根本区别：BIO 是"来一个连接起一个线程"，线程大部分时间阻塞在 IO 等待上；NIO 是"一个线程轮询所有连接的事件"，只有数据就绪才去处理，线程利用率高得多。这也是为什么高并发服务器必须用 NIO 模型。

最后是实际应用。Spring MVC 传统的 Tomcat 是 BIO 模型，Tomcat 8.5+ 支持 NIO 模式；RPC 框架 Dubbo 底层用 Netty（NIO）；项目中的文件上传下载、批量导出通过 IO 流处理时注意用缓冲流（BufferedInputStream）提升性能，大文件用流式处理避免一次性加载进内存。

计算机网络

> 💼 项目锚点速查（背八股时建立项目关联）：
>
> - 三次握手/TCP 连接 → 道富 Druid 连接池建连、券享购网关 Feign 调用
> - TIME_WAIT → 券享购秒杀短连接堆积（keep-alive + 连接池规避）
> - HTTP/1.1 长连接 → 券享购网关 keep-alive 复用连接
> - Cookie/Session/Token → 券享购 Spring Session + Redis 共享 / ZRagent Sa-Token 鉴权
> - 连接池原理 → 道富 Druid 调优（initialSize 1→5、maxActive 10→50，P99 2s→50ms）
> - WebSocket/SSE → ZRagent 流式输出（SSE 服务端主动推送）
> - 粘包/拆包 → 道富流式导出消息头声明长度
> - 反向代理/负载均衡 → 券享购 Gateway `lb://` 路由
> - 状态码 429 → 券享购 Sentinel 限流；502/504 → 道富报表推送网关超时排查

高频问题



Q1. TCP三次握手的过程以及为什么是三次?

面试官您好，TCP 三次握手是为了建立可靠的端到端连接，本质是双方确认彼此的收发能力正常并协商初始序列号。

第一次握手客户端发送 SYN 报文，携带自己的初始序列号；

> **优化补充**：ISN（Initial Sequence Number）必须随机生成，原因有两个：① 防止旧连接的数据包被新连接误认为是合法数据（同一四元组的不同连接）；② 防止序列号被预测，避免伪造报文的安全攻击（如 TCP 序列号预测攻击）。

第二次握手服务器回复 SYN+ACK 报文，携带自己的初始序列号并确认客户端的序列号；

第三次握手客户端再回复 ACK 报文确认服务器的序列号，此时双方都进入连接建立状态。

之所以是三次而不是两次，最主要的原因是防止已失效的连接请求报文导致服务器错误建立连接，浪费资源。同时三次也是确认双方收发能力正常的最小次数，~~而四次则没有必要，因为服务器可以将 SYN 和 ACK 合并在一个报文中发送~~

> **优化补充**：
>
> - 握手前：客户端 `CLOSED` → 发 SYN → `SYN_SENT`
> - 服务端收到 SYN：`LISTEN` → 发 SYN+ACK → `SYN_RCVD`
> - 客户端收到 SYN+ACK：发 ACK → `ESTABLISHED`
> - 服务端收到 ACK：`ESTABLISHED`

Q2. TCP四次挥手的过程以及为什么是四次?

TCP 四次挥手是为了安全地终止全双工连接，本质是双方各自关闭自己的发送通道。

第一次挥手主动关闭方发送 FIN 报文，表示自己没有数据要发送了；

第二次挥手被动关闭方回复 ACK 确认，此时主动方向的连接关闭，但被动方向还可以继续传输数据；

第三次挥手被动关闭方发送完所有数据后，也发送 FIN 报文表示自己也没有数据要发送了；

第四次挥手主动关闭方回复 ACK 确认，双方最终关闭连接。

之所以是四次而不是三次，最核心的原因是 TCP 是全双工协议，被动关闭方收到 FIN 时可能还有数据要发送，所以不能立即发送 FIN，必须先回复 ACK，等数据发送完再发送 FIN，这两个步骤无法合并。

另外，主动关闭方最后会进入 TIME_WAIT 状态等待 2MSL 时间，这是为了确保最后一个 ACK 能到达对方，同时防止旧连接的报文干扰新连接。

> **优化补充**：MSL（Maximum Segment Lifetime）是报文的最大存活时间，Linux 默认是 60 秒，所以 TIME_WAIT 持续 120 秒。选择 2MSL 的原因是：如果最后一个 ACK 丢失，被动方会重发 FIN，主动方需要能在这段时间内收到并重新回复 ACK，一来一回最坏情况就是 2MSL。

Q3. TCP和UDP的区别?

TCP 和 UDP 是传输层的两个核心协议，它们的本质区别在于设计目标不同。TCP 是面向连接的可靠传输协议，提供确认应答 超时重传、序列号、拥塞控制和流量控制机制，保证数据可靠有序地到达，但传输速度较慢，开销较大。UDP 是无连接的不可靠传输协议，不提供可靠性保证，但传输速度极快，开销极小，实时性好。

> - **可靠性**：TCP 通过序列号 + 确认应答 + 超时重传保证数据不丢失不乱序；UDP 不保证，可能丢包、重复、乱序

- **传输方式**：TCP 是字节流，无消息边界（需要处理粘包）；UDP 是数据报，保留消息边界

TCP 适用于对数据完整性要求高的场景，比如文件传输、网页浏览和电子邮件；UDP 适用于对实时性要求高的场景，比如视频通话、直播和在线游戏。另外，现在很多新的协议比如 QUIC 是基于 UDP 实现的，它在应用层增加了可靠性机制，结合了两者的优点。



Q4. TCP的流量控制和拥塞控制

TCP 流量控制和拥塞控制是 TCP 可靠传输的两大核心机制。流量控制解决的是端到端的速度匹配问题，通过滑动窗口协议防止发送方发送过快导致接收方缓冲区溢出；拥塞控制解决的是全局网络的拥塞问题，通过慢启动、拥塞避免、快速重传和快速恢复四个算法，动态调整发送方的发送速率，防止过多数据注入网络导致网络性能下降。

优化补充：

- **慢启动**：连接建立初始 cwnd=1 MSS，每收到一个 ACK，cwnd 加 1，呈指数增长，直到达到慢启动阈值（ssthresh）
- **拥塞避免**：cwnd 超过 ssthresh 后，每个 RTT 只加 1 MSS，线性增长，探测网络上限
- **快重传**：收到 3 个重复 ACK 时立即重传丢失报文，不必等待超时（超时重传代价极大，cwnd 会归 1）
- **快恢复**：快重传后将 ssthresh 设为当前 cwnd 的一半，cwnd 从新 ssthresh 开始线性增长，避免从头慢启动

流量控制的核心是接收窗口 rwnd，拥塞控制的核心是拥塞窗口 cwnd，发送方的实际发送窗口大小是两者的最小值。现在主流的拥塞控制算法是 CUBIC 和 BBR，其中 BBR 基于带宽和延迟来判断拥塞，在高速网络中性能更好。

"为什么慢启动初始 cwnd 是 1，现在实际上是多少？"

RFC 6928（2013年）将初始 cwnd 从 1 提升到 **10 MSS**，主要是因为现代网络带宽已经大幅提升，初始 cwnd=1 在高延迟高带宽链路上启动太慢（每次 RTT 才翻倍），10 MSS 是经过大量真实网络数据统计得出的更合适的初始值。

"HTTP/1.1 keep-alive 和拥塞控制有什么关系？"

TCP 连接复用（keep-alive）能让连接保持在拥塞避免的线性增长阶段，避免每次新建连接都要重新经历慢启动。这也是 HTTP/1.1 相比 HTTP/1.0 性能大幅提升的原因之一。

Q5. token，session，cookie的区别?

Cookie 是客户端存储的小文本，每次请求自动带，但有 4KB 限制和 CSRF 风险；Session 是服务端存储、有状态，安全但分布式要 Redis 共享；Token 是无状态的 JWT 签名令牌，天然支持分布式跨域，但签发了就不能主动失效、Payload 不加密。

1）Cookie：服务器通过响应头下发给浏览器、由浏览器保存的一小段文本数据。

```
原理（四步工作流）：

① 用户首次访问 → 服务器在响应头 Set-Cookie: user=zhang; Max-Age=86400
② 浏览器收到 → 保存到本地（按域名存）
③ 后续每次请求 → 浏览器自动在请求头 Cookie: user=zhang 带上
④ 服务器读取 Cookie → 识别用户
```

大小限制：约 **4KB**；数量有限；**同源限制**（不能跨域携带）。

本质定位：Cookie 只是一个**存储载体**，它本身不含 "你是谁" 的判断逻辑 —— 它存的可能是 Session ID，也可能是 Token。

2）Session：服务端存储的会话档案（接待员的台账）

服务器端保存的用户会话信息。服务器创建 Session 对象，生成唯一 **Session ID** 作为 "钥匙"，通过 Cookie 发给客户端。

```
原理（四步工作流）：

① 用户登录成功 → 服务器创建 Session 对象（存内存/DB），生成唯一 Session ID
② 服务器通过 Set-Cookie: JSESSIONID=xxx 把 Session ID 发给浏览器
③ 浏览器自动带 Cookie → 服务器拿 Session ID 查 Session
④ 查得到 → 已登录；查不到 → 未登录
```

核心特点：

- **有状态**：身份信息**在服务端**，客户端只拿一把 "钥匙"（Session ID）；
- **更安全**：敏感信息不暴露给客户端，Session ID 泄露也只是钥匙（配合过期）；
- **占用服务端内存**：Session 多了要清理（内存泄漏问题，后面 GC 讲过）；

- **分布式痛点（面试必考）**：Session 存在**单台服务器内存**，多台机器不共享 —— 负载均衡下，请求被分到不同机器，登录状态就丢了。

3）Token：无状态加密令牌（防伪通行证）

定义：服务器签发的、由客户端保存的令牌，**服务器不需要存储任何信息**，靠**签名**验证真伪。主流实现是 **JWT**。

JWT 三段结构（必背）：

```
代码块
1  Header.Payload.Signature
2  |       |         |
3  算法    用户信息   用密钥对前两段做签名
4  (HS256) +过期     (防篡改)
5          时间
```

验证流程：服务器收到 Token → 用同一密钥重新计算签名 → 和 Token 里的签名比对 → 一致则通过。**全程不用查数据库 / Redis**—— 这是 "无状态" 的体现。

核心特点：

- **无状态**：服务器不存任何会话信息 → **天然支持分布式、微服务、跨域**；
- **不能主动失效**：Token 一旦签发，到期前一直有效
- **体积大**：Payload 里东西多了每次请求都多传几 KB。

面试官您好，Cookie、Session 和 Token 都是用于解决 HTTP 无状态问题的技术。Cookie 是客户端存储的小文本文件，每次请求自动携带；Session 是服务端存储的用户会话信息，通过 Session ID 关联用户；Token 是服务器生成的无状态加密令牌，包含用户信息和签名。

- **工作流程**：服务端通过 `Set-Cookie` 响应头将数据写入浏览器，浏览器后续请求自动通过 `Cookie` 请求头携带，对应用透明
- **核心限制**：① 大小限制 4KB；② 同源策略，不能跨域传递；③ 每个域名下 Cookie 数量有限（约 50 个）；④ 容易被 CSRF 攻击利用（浏览器自动携带）

它们最核心的区别在于状态性：Session 是有状态的，服务器需要维护会话信息，适合传统单体应用；

- **工作流程**：用户登录后服务端创建 Session 对象（存在内存或数据库），生成唯一 Session ID 通过 Cookie 返回给客户端；后续请求携带 Session ID，服务端查找对应 Session 验证身份
- **分布式痛点**：Session 存在服务端内存，多台服务器之间不共享，负载均衡时用户请求到不同机器会丢失登录状态。解决方案：① Session 粘性（Nginx IP Hash）；② Session 复制（Tomcat 集群广播）；③ 集中存储（Redis 共享 Session），其中 Redis 方案最主流

Token 是无状态的，服务器不需要存储任何信息，天然支持分布式和跨域，适合前后端分离和微服务架构；Cookie 本身只是一个存储载体，通常用来传递 Session ID 或 Token。

- **JWT 三段结构**：`Header.Payload.Signature`，Header 包含算法类型（如 HS256），Payload 包含用户信息和过期时间（Base64 编码，**不是加密**），Signature 用密钥对前两段签名，防止篡改
- **验证流程**：服务端收到 Token 后，用同一密钥重新计算签名与 Token 中的签名比对，一致则验证通过，不需要查数据库或 Redis
- **核心缺点**：① Token 一旦签发无法主动失效（除非引入黑名单机制，又回到了有状态）；② Payload 是 Base64 编码不是加密，敏感信息不能放入；③ Token 体积比 Session ID 大，每次请求都要传输

在安全性方面，Token 通过签名防止篡改，安全性最高；Session 次之；Cookie 存在 CSRF 攻击风险。实际开发中，我们通常会根据应用的架构和安全需求来选择合适的技术，并且都会配合 HTTPS 使用。



Q6. HTTP 1.0 / 1.1 / 2.0 / 3.0 对比

HTTP 协议的四个主要版本都是为了解决前一个版本的性能问题而演进的。演进路径就是 "**连接复用 → 并行传输 → 彻底摆脱 TCP 限制**"：

HTTP/1.0 是第一个正式标准，使用短连接，性能很差；

HTTP/1.1 引入了默认长连接和缓存机制，解决了连接复用问题，成为目前使用最广泛的版本，但存在应用层队头阻塞问题；

代码块

```
客户端                                  服务器
  |  ── GET /a ──────────────────▶|   请求① 处理很慢（3秒）
  |  ── GET /b ──────────────────▶|   请求② 已处理完，响应就绪
  |  ── GET /c ──────────────────▶|   请求③ 已处理完，响应就绪
  |                               |
```

```
6  |                                  |  ⚠️ 但 /b、/c 的响应不能先发！
7  |  ◀── /a 的响应 ──────────────────|  （等了 3 秒，终于发出）
8  |  ◀── /b 的响应 ──────────────────|  /b 响应虽然早就好了，只能排队
9  |  ◀── /c 的响应 ──────────────────|  /c 同理
```

> **优化补充**：HTTP/1.1 相比 1.0 的主要改进：
>
> - **持久连接**（keep-alive）：默认复用 TCP 连接，避免每次请求重新三次握手（性能提升的根本）
> - **管道化**（Pipelining）：允许同一连接连续发多个请求不等响应，但响应必须按顺序返回，这就是**应用层队头阻塞**的根源
> - **Host 头**：支持虚拟主机，同一 IP 可以托管多个域名
> - **分块传输**（Chunked Transfer）：支持流式响应，不需要提前知道 Content-Length
> - **更丰富的缓存控制**：`Cache-Control`、`ETag`、`Last-Modified`

HTTP/2.0 引入了二进制分帧和多路复用，解决了应用层队头阻塞，性能大幅提升，但仍然受限于 TCP 层的队头阻塞；

HTTP/3.0 基于 QUIC 协议（基于 UDP），彻底解决了所有层面的队头阻塞，并且支持 0-RTT 握手和连接迁移，是目前性能最好的 HTTP 版本（**每个流独立传输，丢包只影响自己的流** → 彻底解决队头阻塞）。

现在 HTTP/1.1 和 HTTP/2.0 占据了绝大多数市场份额，~~HTTP/3.0 正在快速普及，未来会成为主流。~~



Q7. HTTP和HTTPS 的区别?

关于 HTTP 和 HTTPS 的区别，我从核心本质、七大核心维度对比、HTTPS 完整握手流程、核心安全机制、常见误区和实际开发应用六个方面来回答：

首先，两者的核心本质和设计目标完全不同。HTTP 是明文传输的超文本传输协议，诞生于 1991 年，设计目标是快速、简单地传输超文本数据，没有任何安全机制；HTTPS 是 HTTP+SSL/TLS 的安全超文

本传输协议，在 HTTP 和 TCP 之间增加了一层 SSL/TLS 加密层，设计目标是解决 HTTP 明文传输的安全问题，保证数据传输的机密性、完整性和身份认证。简单来说：HTTPS 就是穿了 "安全外套" 的 HTTP。

接下来是两者最核心的七大维度对比，这也是面试必背内容：

对比维度	HTTP	HTTPS
传输方式	明文传输，数据直接暴露在网络中	加密传输，所有数据都经过 SSL/TLS 加
默认端口	80	443
安全性	完全不安全，容易被窃听、篡改和冒充	安全，能够有效防止窃听、篡改和中间
性能	更快，不需要加密解密和握手开销	稍慢，有 SSL/TLS 握手和加密解密的性
证书要求	不需要任何证书	必须向 CA 机构申请数字证书，大部分
OSI 层级	应用层协议	应用层 + 传输层之间的 SSL/TLS 层
适用场景	内部系统、静态资源、对安全要求不高的场景	所有涉及用户隐私、支付、交易的互联

然后是 HTTPS 的完整握手流程，这是面试的核心考点。HTTPS 的安全建立在 SSL/TLS 握手之上，以 TLS 1.2 为例，完整握手分为四个步骤：

1. 客户端发送 Client Hello：客户端向服务器发送支持的 TLS 版本、加密套件列表和一个随机数 Client Random
2. 服务器返回 Server Hello：服务器选择双方都支持的 TLS 版本和加密套件，返回服务器随机数 Server Random 和数字证书
3. 客户端验证证书并生成预主密钥：客户端验证证书的合法性，验证通过后生成一个随机的预主密钥 Pre-Master Secret，用服务器公钥加密后发送给服务器
4. 双方协商生成会话密钥：客户端和服务器分别用 Client Random、Server Random 和 Pre-Master Secret 生成相同的会话密钥，之后所有通信都使用这个对称会话密钥加密

整个握手过程只需要两次往返，完成后就进入加密通信阶段。

接下来是 HTTPS 的三大核心安全机制，这也是它能够保证安全的根本原因：

1. 机密性：通过对称加密算法（AES）加密传输的数据，即使被截获也无法解密
2. 完整性：通过哈希算法（SHA-256）生成消息摘要，防止数据在传输过程中被篡改
3. 身份认证：通过数字证书（含公钥、域名、有效期、CA 签名）和非对称加密算法（RSA）验证服务器的身份，防止中间人攻击

> 加密防 "看"（窃听）、摘要防 "改"（篡改）、证书防 "假"（冒充）。

这里我想重点讲一下对称加密和非对称加密的配合使用：非对称加密速度慢，只用来加密传输预主密钥；对称加密速度快，用来加密实际的业务数据。两者结合，既保证了安全性，又兼顾了性能。

然后是常见误区部分，我想澄清三个非常常见的错误认知：第一，很多人以为 HTTPS 会严重影响性能，其实现在的硬件和 TLS 1.3 协议已经将握手开销降到了极低，HTTPS 的性能损失已经可以忽略不计；第二，很多人以为 HTTPS 是绝对安全的，其实不是，如果证书被泄露或者用户点击了钓鱼网站的假证书，仍然会有安全风险；第三，很多人以为只要用了 HTTPS 就不需要其他安全措施了，其实 HTTPS 只能保证传输层的安全，应用层的安全问题（比如 SQL 注入、XSS）仍然需要单独处理。



Q8. HTTPS握手过程说一下

关于 HTTPS 握手过程，我从核心本质、TLS 版本演进、TLS 1.2 完整握手流程、TLS 1.3 重大改进、核心安全机制、常见误区和实际开发应用七个方面来回答：

首先，HTTPS 握手的核心本质是SSL/TLS 协议的安全通道建立过程。它的核心目标是在不安全的公共网络上，通过非对称加密、对称加密和数字证书技术，协商出双方都认可的加密算法和唯一的会话密钥，同时验证服务器的真实身份，最终建立一个机密、完整、可认证的加密通信通道。所有 HTTPS 通信都必须先完成 TLS 握手，才能传输业务数据。

接下来是 TLS 协议的版本演进，这是面试必考点。TLS 是 SSL 的继任者，目前主流使用的是两个版本：

- TLS 1.2：2008 年发布，是目前兼容性最好的版本，绝大多数网站和浏览器都支持，也是过去十年的工业标准
- TLS 1.3：2018 年发布，是最新的稳定版本，握手延迟降低了 50%，安全性大幅提升，移除了所有不安全的加密算法，现在已经成为主流浏览器和服务器的默认版本

- 更早的 SSL 3.0、TLS 1.0、TLS 1.1 都存在严重的安全漏洞，已经被所有主流浏览器废弃，生产环境必须禁用

然后是面试必考的**TLS 1.2 完整握手流程，整个过程需要 2 次 RTT（往返时间），分为四个清晰的阶段：**

第一阶段：客户端发起握手请求（Client Hello）

客户端向服务器发送握手请求，包含以下核心信息：

1. 客户端支持的最高 TLS 版本号
2. 客户端支持的加密套件列表（按优先级排序）
3. 客户端生成的第一个随机数：`Client Random`

第二阶段：服务器响应握手（Server Hello）

服务器收到请求后，返回响应信息，包含以下核心内容：

1. 双方协商确定的 TLS 版本号（取双方都支持的最高版本）
2. 从客户端列表中选择的加密套件
3. 服务器生成的第二个随机数：`Server Random`
4. 服务器的数字证书（包含服务器公钥、域名、有效期、CA 签名等信息）
5. 服务器 Hello Done 消息，表示服务器端的 Hello 阶段结束

第三阶段：客户端验证证书并发送密钥材料

客户端收到证书后，首先进行严格的证书验证：

1. 验证证书是否由可信的 CA 机构签发
2. 验证证书的域名是否与当前访问的域名一致
3. 验证证书是否在有效期内
4. 验证证书是否被吊销

验证通过后，**客户端生成第三个随机数**：`Pre-Master Secret`（预主密钥），用服务器公钥加密后发送给服务器。

> Pre-Master Secret 是整个密钥体系的 "核心秘密"

```
① Client Random：客户端发 Client Hello 时明文带上 → 服务器看到了 ✓
② Server Random：服务器回 Server Hello 时明文带上 → 客户端看到了 ✓
③ Pre-Master Secret：客户端生成后，用【服务器公钥】加密发出
    → 只有服务器用【私钥】能解开 → 服务器拿到 ✓（中间人拿到也解不开）
```

第四阶段：双方协商生成会话密钥并完成握手

1. 客户端和服务器分别使用之前的三个随机数（Client Random、Server Random、Pre-Master Secret），通过相同的密钥派生算法，生成完全相同的会话密钥
2. 客户端发送 `Change Cipher Spec` 消息，表示后续所有通信都将使用会话密钥加密
3. 客户端发送 `Finished` 消息，包含之前所有握手消息的哈希值，供服务器验证
4. 服务器同样发送 `Change Cipher Spec` 和 `Finished` 消息
5. 双方验证 Finished 消息通过后，握手正式完成，开始传输加密的业务数据

接下来是TLS 1.3 的重大改进，这是现在面试的热点问题。TLS 1.3 对握手流程进行了革命性的优化：

1. 握手延迟减半：从 TLS 1.2 的 2 次 RTT 减少到 1 次 RTT，大幅提升了页面加载速度
2. 0-RTT 握手：对于已经建立过连接的客户端，可以直接发送加密的业务数据，不需要等待握手完成
3. 安全性大幅提升：移除了所有不安全的加密算法和密钥交换方式，只保留了 5 个安全的加密套件
4. 简化握手流程：合并了多个握手消息，减少了网络传输量

这里我想重点讲一下三个随机数的作用：为什么需要三个随机数而不是一个？因为如果只用一个随机数，一旦这个随机数被破解，整个会话的密钥就会被破解。使用三个独立的随机数，分别由客户端和服务器生成，即使其中一个被破解，也不会影响整个会话的安全性，极大地提高了密钥的随机性和不可预测性。

然后是常见误区部分，我想澄清三个非常常见的错误认知： 第一，很多人以为握手过程中会传输会话密钥，其实不是，会话密钥永远不会在网络上传输，而是由双方各自独立生成的 第二，很多人以为 HTTPS 会验证客户端的身份，其实默认的 HTTPS 握手只验证服务器的身份，客户端身份验证是可选的双向认证 第三，很多人以为 TLS 1.3 的 0-RTT 握手是绝对安全的，其实 0-RTT 存在重放攻击的风险，不能用于传输敏感数据

最后是实际开发中的应用。在项目中，我会严格遵循 HTTPS 的最佳实践：

1. 强制使用 TLS 1.2 和 TLS 1.3，禁用所有旧版本协议
2. 只启用安全的加密套件，禁用所有不安全的算法
3. 配置证书自动续期，避免证书过期导致服务不可用
4. 开启会话复用（Session Ticket），减少重复握手的开销
5. 开启 OCSP 装订，提高证书验证的速度和隐私性
6. 配置 HSTS 头，强制浏览器使用 HTTPS 访问，防止降级攻击

代码块

```
① 客户端：Client Hello（版本、套件、Client Random）
② 服务器：Server Hello（Server Random）+ 证书
③ 客户端：验证证书 → 生成 Pre-Master Secret → 用公钥加密发出
④ 服务器：私钥解密 → 拿到 Pre-Master Secret
⑤ 双方各自：三随机数 → 派生【相同】会话密钥    ← 你问的⑤：密钥是"算"出来的，从不上网
⑥ 客户端：ChangeCipherSpec（明文预告"我切了"）
```

```
            → Finished（加密"验货"，用新密钥，内容是握手摘要）
⑦ 服务器：收到并解密 Finished，验证通过 → 也发 ChangeCipherSpec + Finished
            → 客户端解密验证通过 → 握手完成
之后：全部用会话密钥（AES）加密通信
```

Q9. RPC 和 HTTP 有什么区别?

> **RPC 是设计思想、HTTP 是协议**

HTTP更像是对外暴露接口的通用方式，RPC更像是服务内部互相调用的方法调用方式。先说HTTP，HTTP本质上是一种应用层通信协议，平时写的接口，例如用户登录、查询订单、提交订单等都是通过HTTP完成的，它的特点是通用性强，跨语言、跨平台、跨系统都很方便，比如前端调用后端，浏览器访问服务器，等都会使用HTTP，因为接入成本低，易于调试；再说RPC，全称远程核心调用，它的核心目标是：让你调用远程服务，就像调用本地方法一样，无需关心底层怎么建立连接、怎么序列化、怎么传输、怎么拿返回值，这些事情RPC框架会帮我们进行封装。

那他们最大的区别是什么?

> " 最核心的区别是**使用场景**—— 对外用 HTTP，对内用 RPC。对外看重通用性和兼容性，HTTP 用 JSON 可读性好、跨语言；对内看重性能和治理，RPC 用高效序列化体积小，框架自带负载均衡、熔断、服务发现。一句话：对外用 HTTP 接口，对内用 RPC 调用，本质是**通用性 vs 性能治理**的取舍。"

1. 使用场景不同，HTTP更适合对外接口，比如开放平台、前后端交互、第三方系统对接；RPC更适合内部服务调用，比如微服务之间的调用；对外接口更看重通用性和兼容性，内部调用更看重性能、治理能力和开发体验
2. 调用方式不同，HTTP通常是面向资源的，比如查询用户，就是访问用户相关接口，它强调的是我对某个资源做什么操作；而RPC，更像是面向方法的，比如查询库存方法，它强调的是，我调用远程服务里的某个函数，所以RPC写起来更像是本地方法调用
3. 数据传输方式不同，HTTP的数据传输方式常见是JSON，JSON的优点是可读性好，调试方便，缺点是体积相对大，解析成本高；而RPC框架通常会使用更高效的序列化方式，优点是传输体积小，解析速度快，缺点就是可读性没那么强，在内部调用场景下，RPC往往性能更好
4. 服务治理能力不同，普通HTTP调用，本身只是一次请求和响应，如果想做负载均衡、熔断、限流、重试、服务发现、链路追踪通常要额外接入网关、注册中心或者中间件，而成熟的RPC框架往往就会集成这些能力，这也是微服务内部选择RPC的原因

网络分层模型

1.1 OSI 七层模型 vs TCP/IP 四层模型

```
代码块
┌──────────────────┐    ┌──────────────────┐    ┌──────────────────────────┐
│   OSI 七层       │    │  TCP/IP 四层     │    │   谷粒商城中的体现       │
├──────────────────┤    ├──────────────────┤    ├──────────────────────────┤
│  7 应用层        │──→ │  4 应用层        │    │ HTTP（浏览器→网关）      │
│  6 表示层        │    │                  │    │ HTTPS/TLS                │
│  5 会话层        │    │                  │    │                          │
├──────────────────┤    ├──────────────────┤    ├──────────────────────────┤
│  4 传输层        │──→ │  3 传输层        │    │ TCP（Feign调用）         │
│                  │    │                  │    │ UDP（DNS/视频）          │
├──────────────────┤    ├──────────────────┤    ├──────────────────────────┤
│  3 网络层        │──→ │  2 网络层        │    │ IP（路由转发）           │
├──────────────────┤    │                  │    │ ICMP/ARP                 │
│  2 数据链路层    │──→ │  1 网络接口层    │    │ MAC/VLAN                 │
├──────────────────┤    │                  │    ├──────────────────────────┤
│  1 物理层        │    │                  │    │ 网卡/光纤/交换机         │
└──────────────────┘    └──────────────────┘    └──────────────────────────┘
```

面试追问：

> **Q：为什么需要分层？**
>
> 1. **解耦**：每一层只关心自己的功能，上层调用下层，不关心实现细节
> 2. **标准化**：接口统一，每层可以独立演进
> 3. **复用**：下层服务可以被多个上层服务复用

TCP 协议（传输控制协议）

2.1 TCP 三次握手

> **是什么**：建立可靠连接的过程，双方确认彼此的发送和接收能力正常。

```
代码块
    三次握手：
客户端                                服务端
   │                                    │
   │ ——— SYN=1, seq=x ————————————▶ │   ① 客户端发送 SYN（同步报文）
   │                                    │
   │ ◀—— SYN=1, ACK=1, seq=y ——— │   ② 服务端返回 SYN+ACK
   │       ack=x+1                      │
   │                                    │
   │ ——— ACK=1, seq=x+1 ————————————▶ │   ③ 客户端发送 ACK
   │       ack=y+1                      │
   │                                    │
   │        连接建立完成                 │
   │                                    │
```

```
14      │ ——— 数据传输 ————————————▶ │
15      │ ◀——— 数据传输 ———————————— │
```

2.2 TCP 四次挥手

是什么：断开连接的过程，双方各自关闭发送通道。

代码块

```
1     客户端                                    服务端
2       │                                        │
3       │  ————————— FIN=1，seq=u ————————————→  │   客户端：我发送完毕，请求关闭
4       │  ←————————— ACK=1，ack=u+1 ————————————│   服务端：收到，我还在发送数据
5       │  ←——— 关闭写端，发送FIN=1，seq=w ——|   服务端：我发送完毕，请求关闭
6       │  ————————— ACK=1，ack=w+1 ————————————→ |   客户端：收到，我知道了
7       │                                        │
8       │        TCP 连接彻底关闭                  │
```

为什么需要四次：TCP 是全双工协议，每个方向需要单独关闭

Time_WAIT 状态（客户端）：

- 等待 2MSL（Maximum Segment Lifetime，通常 2 分钟）
- 目的：1. 确保最后的 ACK 能到达服务端；2. 让本连接的所有报文在网络中消失

2.3 TCP 可靠传输原理

滑动窗口：

- 发送方维护一个窗口，窗口内的数据可以连续发送，无需等待 ACK
- 收到 ACK 后窗口滑动

拥塞控制：

- `cwnd`（拥塞窗口）动态调整
- 慢启动：`cwnd` 从 1 开始指数增长，到达阈值后线性增长
- 快重传：收到 3 个重复 ACK 直接重传
- 快恢复：拥塞避免

2.4 TCP 与 UDP 的区别

对比项	TCP	UDP
连接性	面向连接	无连接
可靠性	可靠（有 ACK、重传）	不可靠（不保证到达）
效率	低（有确认、重传开销）	高
传输方式	字节流	数据报
适用场景	文件传输、HTTP、邮件	DNS、实时音视频、游戏
首部开销	20-60 字节	8 字节
拥塞控制	有	无

面试追问：

> **Q：为什么 DNS 用 UDP?**
>
> DNS 查询通常很小（几十字节），UDP 效率更高。而且 DNS 有重试机制，丢包就重发。大型 DNS 会用 TCP 做区域传输。

HTTP 协议（超文本传输协议）

3.1 HTTP 1.0 / 1.1 / 2.0 / 3.0 对比

版本	关键特性	问题
HTTP 1.0	短连接，每个请求建立一个 TCP 连接	连接频繁建立销毁开销大
HTTP 1.1	**持久连接**（Connection: keep-alive）、管道化	队头阻塞（HOL Blocking）
HTTP 2.0	**多路复用**（一个连接并行多个请求）、二进制分帧、头部压缩	TCP 队头阻塞（仍存在）
HTTP 3.0	**QUIC**（基于 UDP）、无队头阻塞、0-RTT	部署成本高

项目中： `gulimall-gateway` 和微服务之间默认 HTTP 1.1（Spring Cloud 默认配置），可通过配置升级到 HTTP 2.0

3.2 HTTP 请求方法

方法	含义	项目中的例子
GET	查询资源	/toTrade、/list.html
POST	提交资源	/submitOrder、/register
PUT	更新资源	更新订单状态
DELETE	删除资源	删除购物车商品
HEAD	获取头部	curl -I 检查资源

GET vs POST 区别：

- GET 参数在 URL 中，有长度限制（浏览器约 2KB），POST 在 body 中无限制
- GET 请求会被浏览器缓存，POST 不会
- GET 是幂等的（多次请求结果相同），POST 不幂等

3.3 HTTP 状态码

状态码	含义	项目中的例子
200	成功	R.ok() → 前端收到 200
302/301	重定向	redirect:
400	参数错误	验证码错误
401	未认证	Session 无用户信息
403	无权限	
404	资源不存在	
500	服务器内部错误	运行时异常
502	网关错误	Gateway 无法连接到后端服务

DNS 域名解析

4.1 DNS 查询过程

```
浏览器缓存 → 系统缓存（hosts）→ 本地 DNS 缓存 →
    ↓
递归查询：本地 DNS 服务器（运营商）
    ↓（如果本地 DNS 没有）
迭代查询：根域名服务器 → .com 顶级域 → gulimall.com 权威 DNS
```

解析记录类型：

- `A` 记录：将域名解析为 IPv4 地址
- `AAAA` 记录：IPv6 地址
- `CNAME`：别名
- `MX`：邮件交换记录
- `TXT`：SPF 反垃圾邮件

项目中： `gulimall.com`、`auth.gulimall.com` 等域名通过 DNS 解析到 Nginx/Gateway IP

4.2 负载均衡与 DNS

```
用户请求 gulimall.com
          ↓
DNS 返回不同 IP（轮询/地理位置/权重）
          ↓
不同的 Nginx 服务器/Gateway 集群
```

网关与反向代理

5.1 正向代理 vs 反向代理

对比项	正向代理	反向代理
代理对象	客户端	服务端
典型例子	VPN、科学上网	Nginx、API Gateway
隐藏	隐藏客户端	隐藏服务端
客户端知道	知道目标服务器	不知道真实服务端

Spring Cloud Gateway（反向代理）：

```
用户浏览器 → http://auth.gulimall.com → DNS → Nginx → Gateway
                                                        ↓
                                                  Nacos 服务发现
                                                        ↓
                                                  路由到具体服务（lb://gulimall-auth-server）
```

5.2 负载均衡

客户端负载均衡（Ribbon）：

- 负载均衡器在客户端，Ribbon 从注册中心获取服务列表，根据策略选择
- 项目中 `gulimall-gateway` 通过 `lb://` 前缀实现 Ribbon 负载均衡

服务端负载均衡（Nginx）：

- Nginx 作为反向代理服务器，根据配置分发请求
- 支持轮询、权重、IP Hash、最少连接数

服务端的负载均衡算法：

- 轮询（Round Robin）
- 随机（Random）
- 加权轮询（Weighted Round Robin）
- 最少活跃数（Least Connections）
- 一致性哈希（Consistent Hash）

项目中： `lb://gulimall-product` 使用 Ribbon 默认轮询算法

常见问题：

Q3. TCP 的滑动窗口和拥塞控制⭐⭐⭐⭐

滑动窗口（流量控制）：

代码块

```
发送方窗口（已发送未确认 + 可以发送）：
┌──────────────────────────────────────────────────────────┐
│ 已确认  | 已发送未确认 | 可发送(窗口内) | 不可发送  |
│ 0~500   |  501~700    |   701~900    |  901~    |
└──────────────────────────────────────────────────────────┘
            ↑
          窗口左边界向右滑动

接收方窗口：
┌──────────────────────────────────────────────────────────┐
│ 已接收已确认 | 期望接收(窗口内)  | 不期望接收       |
│  0~500       |   501~700       |  701~          |
└──────────────────────────────────────────────────────────┘
            ↑
          窗口左边界向右滑动

接收方通过 ACK 告知发送方自己的窗口大小（rwnd）
→ 发送方不能发送超过接收方窗口的数据
→ 防止发送方发送速度超过接收方处理速度
```

拥塞控制（防止网络过载）：

代码块

```
拥塞窗口（cwnd）动态调整：

慢启动阶段：
```

```
    cwnd = 1 MSS
    每收到一个 ACK → cwnd += 1 MSS
    → cwnd 指数增长：1, 2, 4, 8, 16...
    → 直到 cwnd 达到 ssthresh（慢启动阈值）

拥塞避免阶段（cwnd > ssthresh）：
    每收到一个 ACK → cwnd += 1/cwnd
    → cwnd 线性增长

丢包检测：
  超时 → ssthresh = cwnd/2, cwnd = 1（重新慢启动）
  3次重复ACK → ssthresh = cwnd/2, cwnd = cwnd/2（快速恢复）
```

Q5. HTTPS 的工作原理⭐⭐⭐⭐⭐

代码块

```
TLS 握手过程（RSA 密钥交换）：
客户端                                                    服务端
  |                                                         |
  | ——— ClientHello（支持的TLS版本，加密套件，随机数）——▶| ①
  | ◀—— ServerHello（选定的TLS版本，加密套件，随机数）——| ②
  | ◀—— Certificate（服务器证书链）————————————————| ③
  | ◀—— ServerHelloDone ————————————————————————————|
  |                                                         |
  | ——— ClientKeyExchange ——————————————————————————▶| ④
  |     (用服务端公钥加密的 PreMasterSecret)                  |
  | ——— ChangeCipherSpec ———————————————————————————▶| ⑤
  | ——— Finished（握手摘要）————————————————————————▶| ⑥
  | ◀—— ChangeCipherSpec ————————————————————————————| ⑦
  | ◀—— Finished（握手摘要）—————————————————————————| ⑧
  |                                                         |
  |          双方用 PreMasterSecret 派生对称密钥              |
  |                                                         |
  | ——— 应用数据（加密）————————————————————————————▶|
  | ◀—— 应用数据（加密）————————————————————————————|
```

为什么不直接用非对称加密传输数据：

- 非对称加密（RSA 2048位）：比对称加密（RC4/AES）慢 **100~1000 倍**
- HTTPS 用非对称加密**交换对称密钥**，之后用对称密钥加密实际数据

Q6. 连接池（数据库/HTTP/Redis）的原理

连接池的核心机制：

代码块

```
应用启动
    ↓
连接池初始化：建立 initialSize=5 个连接
    ↓
请求到来
    ↓
从池中借出连接（getResource()）
    ↓
业务逻辑使用连接（查询/写入）
    ↓
归还连接到池（close() → 归还池，不是真正关闭）
    ↓
后台线程定时检测空闲连接健康性（testWhileIdle）
    ↓
应用关闭时：关闭所有连接（destroy-method="close"）
```

为什么连接要复用：

代码块

```
每次新建连接（无池）：
    建立 TCP 连接：1.5 RTT（30~100ms）
    TLS 握手（如 HTTPS）：1~2 RTT（如果数据库用SSL）
    数据库认证：0.5 RTT
    总计：每个请求多花 50~200ms

使用连接池：
    首次建立连接（池预热）：50~200ms
    后续请求：从池中借出：0.1~1ms
    总计：每个请求几乎无额外开销
```

Q7. JDBC 的 `Connection` 是不是线程安全的？

不是线程安全的。

代码块

```
// 错误用法：多线程共用一个 ConnectionConnection conn =
dataSource.getConnection();
Thread A: conn.setAutoCommit(false); conn.insert();
Thread B: conn.commit();  // ← 并发修改事务状态，数据混乱
```

正确的多线程连接管理：

代码块

```
// 方式1：每个线程一个 Connection（从池中各自借出）
Connection conn1 = dataSource.getConnection();  // Thread A
Connection conn2 = dataSource.getConnection();  // Thread B
// 各自独立，互不干扰

// 方式2：同一个 Connection，用 synchronized 包裹事务操作
synchronized (conn) {
    conn.setAutoCommit(false);
    conn.insert();
    conn.commit();
}

// 项目中的做法：每次查询从池中借出独立的 Connection
try (SqlSession session = sqlSessionFactory.openSession()) {
    // session 持有 Connection
    // 业务逻辑
} // 关闭 session → 归还连接到池
```

Q8. CDN 的工作原理

代码块

```
用户请求 www.example.com/logo.png
    ↓
DNS 解析：域名 → CDN 专用域名
    ↓
DNS 智能调度：最近/最快/最空闲的 CDN 节点
    ↓
CDN 边缘节点检查缓存：
    ├── 命中（Cache Hit）：直接返回缓存内容
    └── 未命中（Cache Miss）：回源（回源站拉取）→ 缓存 → 返回
    ↓
响应给用户（从 CDN 节点，不是源站）
```

CDN 的缓存策略：

- 静态资源（图片/CSS/JS）：长期缓存（`Cache-Control: max-age=31536000`）
- 动态内容：不缓存或短缓存

Q9：浏览器中输入 URL 到页面显示的完整过程？

代码块

```
1. URL 解析 → 提取协议、域名、路径
2. DNS 解析：浏览器缓存 → 系统缓存 → hosts 文件 → DNS 服务器
   域名 → IP 地址（A 记录 / CNAME）
3. 建立 TCP 连接（HTTP）或 TLS 握手（HTTPS）
4. 发送 HTTP 请求
   GET /index.html HTTP/1.1
   Host: www.example.com
   ...
5. 服务器处理请求，返回 HTTP 响应
   HTTP/1.1 200 OK
   Content-Type: text/html
   ...
   <html>...</html>
6. 浏览器解析 HTML → 构建 DOM 树
7. 遇到 <link> → 加载 CSS
8. 遇到 <script> → 执行 JS（可能阻塞渲染）
9. 构建渲染树 → 布局 → 绘制
10. 页面显示
```

操作系统

Q1. 进程，线程，协程的区别是什么？

> " 进程、线程、协程本质是**不同粒度的并发执行单位**，区别在三点：**资源拥有**（进程独立、线程共享进程、协程共享线程）、**调度者**（内核 vs 用户态）、**切换开销**（进程最重、线程中、协程几乎为零）—— 而演进正是痛点驱动的：进程太重所以有了线程，线程切换贵所以有了协程。"

" 我理解这三者的本质是**不同粒度的并发执行单位**，核心区别在**资源拥有、调度者、切换开销**三个维度上。

进程是操作系统**资源分配**的基本单位，有独立的地址空间（页表、文件描述符等），进程之间完全隔离，通信要靠 IPC；缺点是**重**—— 创建要分配独立地址空间，切换时要换页表。

线程是**CPU 调度**的基本单位，是进程内的执行流，**共享进程的资源**，通信靠共享内存加锁；它比进程轻，但**切换还是要陷入内核态**，由内核调度器选下一个线程，保存恢复寄存器，同时线程栈默认就要 1MB，创建几万个线程内存就顶不住了。

协程是**用户态自己调度的**轻量级执行流，也叫纤程或虚拟线程。它不归内核管，**切换是纯用户态的**—— 程序自己决定切换点（yield/await），只保存恢复少量寄存器，**不需要陷入内核态**，所以切换开销接近零，一个线程内可以挂几十万上百万个协程。

演进逻辑其实就是**痛点驱动**：进程太重，所以有了共享资源的线程；线程切换要进内核、栈开销大，所以有了用户态调度的协程。选型上，**需要隔离和稳定性用进程，通用并发用线程，超高并发 IO 密集场景用协程**。

> 什么时候用哪个？
>
> "**隔离 + 稳定**（跨模块、要防崩）→ 进程；**通用并发**（中等数量、要 CPU 并行）→ 线程；**IO 密集型超高并发**（几十万连接、请求频繁阻塞等待）→ 协程 / 虚拟线程。"

具体到 Java：Java 线程是 **1:1 映射内核线程**，每个 Java 线程背后就是一个内核线程；传统上没有原生协程，但 **JDK 21 正式发布了虚拟线程（Project Loom）**，由 JVM 在用户态调度，可以把几十万任务跑在几十个载体线程上，解决高并发 IO 场景下线程池撑不住的问题。Go 的 goroutine、Kotlin 协程也是类似的思路。"

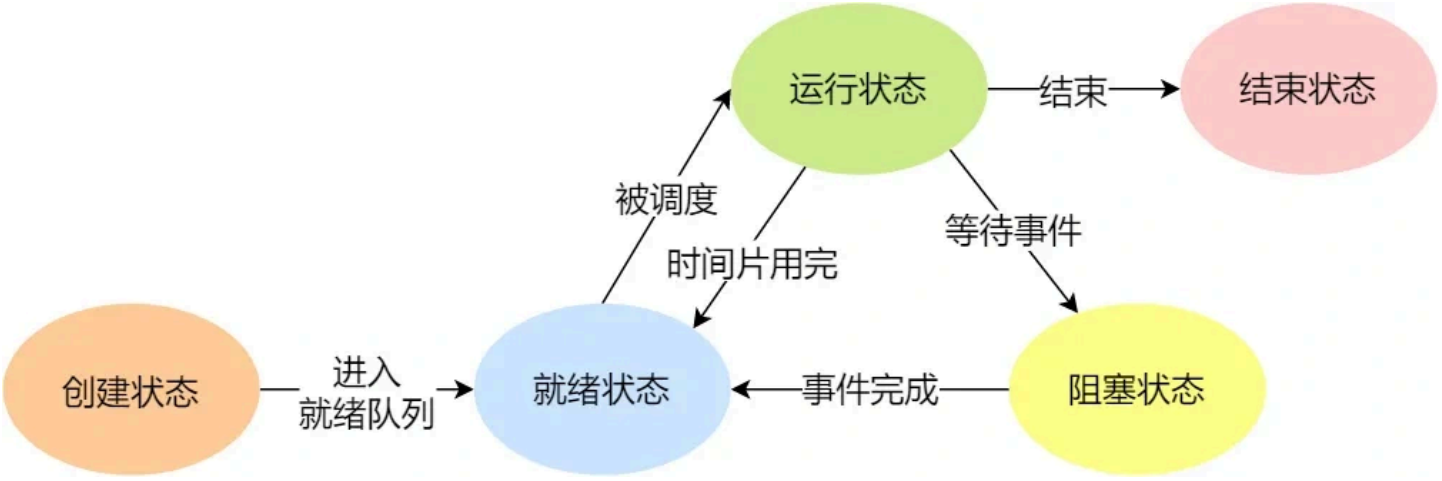
> 对比 Go 是 **M:N 模型**：M 个 goroutine 映射到 N 个内核线程上，运行时自己调度。

多线程是不是越多越好，太多会有什么问题？

多线程不一定越多越好，过多的线程可能会导致一些问题。

- 切换开销增加
- 内存占用增加
- 锁竞争增加

Q2. 进程的状态，如何切换



" 进程的五种经典状态是：**新建（New）、就绪（Ready）、运行（Running）、阻塞（Blocked/Waiting）、终止（Terminated）**。

新建是进程正在创建、还没进入就绪队列；**就绪**是万事俱备，只等 CPU—— 它在就绪队列里排队；**运行**是正在 CPU 上执行；**阻塞**是进程在等待某个事件（IO 完成、等待资源、请求信号量），即使给它 CPU 它也执行不了；**终止**是进程执行结束或被杀死，正在回收资源。

状态转换有六种，关键是**谁触发**：

① **新建 → 就绪**：创建完成，进入就绪队列；

② **就绪 → 运行**：被调度器选中（dispatch），这是**唯一主动获得 CPU 的入口**；

③ **运行 → 就绪**：**时间片耗尽被抢占**，或者更高优先级进程到来（preempt）；

④ **运行 → 阻塞**：进程**主动**请求 IO、等待资源或 sleep—— 注意这是进程自己的行为，不是被强制的；

⑤ **阻塞 → 就绪**：**等待的事件完成**（IO 中断处理完、资源可用）—— 进程被唤醒，重新进入就绪队列；

⑥ **运行 → 终止**：正常结束、异常退出或被 kill。

这里要特别强调**三个不存在的转换**，面试官经常挖坑：**阻塞不能直接到运行**（必须经过就绪队列重新排队）、**新建不能直接到运行**（必须先就绪）、**就绪不能直接到阻塞**（就绪只等 CPU，不会自己跑去等待事件）。

如果再深入一层，还有**七状态模型**，加的是**就绪挂起和阻塞挂起**—— 把进程从内存换出到外存（swap），原因是内存不够、或者调试 / 观察需要。挂起态下多两条转换：就绪 → 就绪挂起（换出）、阻塞 → 阻塞挂起（换出），以及对应的换入。

具体到 Java，线程状态和进程状态有对应关系：Java 的 **NEW 对应新建，TERMINATED 对应终止，RUNNABLE 是就绪 + 运行的合并（JVM 视角没法区分），BLOCKED/WAITING/TIMED_WAITING 对应阻塞**—— 区别是 Java 把 "等待锁"（BLOCKED）和 "主动等待"（WAITING/TIMED_WAITING）分开计了。"

时间片到了会怎样？和主动 sleep 有什么区别？

> " 时间片到是**被抢占**（运行→就绪，被动）；sleep 是**主动让出**（运行→阻塞，自愿）。被抢占的进程依然 ' 可运行 '，只是没 CPU；sleep 的进程是 ' 不可运行 '，要到点才醒。区别就是**被动 vs 主动**、**可运行 vs 不可运行**。"

（注：部分内容由豆包工作 AI 生成）